PANDAS MASTER GUIDE -- ARCHITECTURE

Structure Overview

This guide is divided into 9 modules (some split into a/b for readability, ~300 lines each).

```
pandas-guide/
├── README.md                                ← Overview + navigation
├── MODULE-00-GETTING-STARTED.md             ← Setup, philosophy, core objects
├── MODULE-01a-DATA-INGESTION-CSV-JSON.md    ← Loading CSV and JSON
├── MODULE-01b-DATA-INGESTION-EXCEL-SQL-API.md  ← Loading Excel, SQL, APIs, Parquet
├── MODULE-02a-DATA-EXPLORATION-BASIC.md     ← First look, sampling, summary stats
├── MODULE-02b-DATA-EXPLORATION-ADVANCED.md  ← Correlation, distributions, EDA function
├── MODULE-03a-DATA-CLEANING-MISSING.md      ← Missing values: detection and imputation
├── MODULE-03b-DATA-CLEANING-PATTERNS.md     ← Duplicates, types, strings, outliers
├── MODULE-04a-DATA-MANIPULATION-INDEXING.md ← Indexing, filtering, sorting, columns
├── MODULE-04b-DATA-MANIPULATION-GROUPBY-MERGE.md ← GroupBy, merge, concat
├── MODULE-05a-DATA-TRANSFORMATION-PIVOT-RESHAPE.md ← Pivot tables, wide↔long reshaping
├── MODULE-05b-DATA-TRANSFORMATION-STRINGS-TIMESERIES.md ← Strings, DateTime, time series, categories
├── MODULE-06a-VISUALIZATION-BASIC.md        ← Line, bar, histogram charts
├── MODULE-06b-VISUALIZATION-ADVANCED.md     ← Scatter, box, heatmap, dashboards
├── MODULE-07a-DATA-EXPORT-CSV-EXCEL.md      ← CSV and styled Excel export
├── MODULE-07b-DATA-EXPORT-JSON-SQL-PARQUET.md  ← JSON, SQL, Parquet, production exporter
├── MODULE-08a-PRODUCTION-PERFORMANCE.md     ← Vectorization, memory, chunking
├── MODULE-08b-PRODUCTION-PATTERNS.md        ← Error handling, testing, best practices
├── charts/                                  ← Generated chart images
│   ├── chart01_revenue_trend.png
│   ├── chart02_revenue_by_region.png
│   ├── chart03_revenue_distribution.png
│   ├── chart04_stacked_bar.png
│   ├── chart05_scatter_units_revenue.png
│   ├── chart06_boxplot_region.png
│   ├── chart07_correlation_heatmap.png
│   └── chart08_dashboard.png
└── (total: ~3,600 lines across 18 files)
```

Module Breakdown

MODULE-00: GETTING STARTED

- Pandas philosophy and why it exists
- Core objects: Series, DataFrame, Index
- Installation and version management
- First DataFrame: creating from scratch

MODULE-01: DATA INGESTION

- CSV: read_csv deep dive (all important parameters)
- JSON: read_json, nested JSON handling
- Excel: read_excel, multi-sheet, formatting
- SQL: read_sql, read_sql_query, connections
- APIs: fetching JSON from REST endpoints
- Parquet and other binary formats

- Error handling and chunking for large files

MODULE-02: DATA EXPLORATION (Simple EDA)

- First look: head(), tail(), sample()
- Structure: info(), shape, columns, dtypes
- Summary statistics: describe()
- Value counts and distributions
- Correlation analysis
- Missing data overview
- Quick visual exploration

MODULE-03: DATA CLEANING

- Missing values: detection, analysis, strategies
- Imputation methods (mean, median, forward fill, interpolation)
- Duplicate detection and removal
- Type conversion and validation
- String cleaning and normalization
- Outlier detection and handling
- Data validation patterns

MODULE-04: DATA MANIPULATION

- Indexing and selection: loc, iloc, at, iat
- Boolean filtering and masking
- Sorting (single/multi column, ascending/descending)
- Column operations: add, drop, rename, reorder
- Apply functions: apply, applymap, map
- GroupBy operations: split-apply-combine
- Merge and join: inner, outer, left, right
- Concat and append

MODULE-05: DATA TRANSFORMATION

- Pivot tables and crosstabs
- Melting and stacking (wide to long)
- String operations (str accessor)
- DateTime operations
- Time series resampling
- Window functions (rolling, expanding, exponential)
- Categorical data handling
- Feature engineering basics

MODULE-06: VISUALIZATION

- Matplotlib fundamentals for pandas
- Built-in pandas plotting

- Professional line charts
- Bar charts (grouped, stacked)
- Histograms and density plots
- Scatter plots and bubble charts
- Box plots and violin plots
- Heatmaps and correlation matrices
- Time series visualization
- Subplots and multi-panel layouts
- Chart customization (colors, fonts, labels)
- Production-grade chart templates

MODULE-07: DATA EXPORT

- CSV export: to_csv with formatting options
- Excel export: to_excel with styling (openpyxl, xlsxwriter)
- JSON export: to_json formats
- SQL export: to_sql
- Parquet export
- Multi-sheet Excel reports
- Formatted Excel reports with charts

MODULE-08: PRODUCTION-GRADE

- Performance optimization
- Memory management
- Vectorization vs apply
- Chunking for large datasets
- Error handling patterns
- Logging and monitoring
- Testing data pipelines
- Reproducible workflows
- Best practices checklist

Navigation

Each file is self-contained but references others. Start from MODULE-00 and progress sequentially, or jump to specific modules as needed.

Prerequisites

- Python 3.8+
- Basic Python knowledge (variables, functions, loops)
- No prior pandas experience required (but helpful)

Libraries Used

- pandas (core)

- numpy (numerical operations)
- matplotlib (plotting)
- seaborn (statistical visualization)
- openpyxl (Excel reading/writing)
- xlsxwriter (Excel formatting)
- sqlalchemy (SQL operations)
- requests (API calls)

MODULE-00: GETTING STARTED WITH PANDAS

Why Pandas Exists

Pandas was created by Wes McKinney in 2008 while working at quantitative trading firms. He needed a tool that combined:

- The data manipulation power of Excel/SQL
- The flexibility of Python
- The performance of compiled languages

Why pandas over pure Python lists/dicts?

- Vectorized operations (no slow Python loops)
- Built-in handling of missing data (NaN)
- Labeled axes (column names, row indices)
- Time-series functionality
- Seamless integration with numpy, matplotlib, scipy

Installation

```
# Minimal installation
pip install pandas

# Recommended full installation
pip install pandas numpy matplotlib seaborn openpyxl xlsxwriter sqlalchemy requests

# Verify installation
python -c "import pandas; print(pandas.__version__)"
```

Version note: This guide targets pandas 2.x. Some behaviors differ in 1.x.

Core Philosophy

Pandas is built on three principles:

1. **Labeled data** -- Every row and column has a name (index)
2. **Missing data is first-class** -- NaN values are handled gracefully
3. **Vectorization** -- Operations apply to entire columns at once (fast)

```
import pandas as pd
import numpy as np

# Standard import convention -- always use these aliases
```

```
# pd = pandas (the library)
# np = numpy (numerical computing foundation)
```

The Three Core Objects

1. Series -- 1D Labeled Array

A Series is like a column in a spreadsheet. It has:

- **Values** (the data)
- **Index** (labels for each value)

```
# Creating a Series
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
print(s)
# a    10
# b    20
# c    30
# d    40
# dtype: int64

# Access by label
print(s['a'])      # 10
print(s[['a', 'c']])  # Multiple values

# Access by position
print(s.iloc[0])   # 10 (first position)
print(s.loc['a'])  # 10 (by label)
```

Why index matters: The index lets you align data automatically during operations.

```
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([4, 5, 6], index=['b', 'c', 'd'])
print(s1 + s2)  # Automatic alignment by index
# a    NaN  (only in s1)
# b    6.0  (1 + 5)
# c    8.0  (2 + 6)
# d    NaN  (only in s2)
```

2. DataFrame -- 2D Labeled Table

A DataFrame is like a spreadsheet or SQL table. It's a collection of Series that share an index.

```
# Creating a DataFrame from dict
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'salary': [50000, 60000, 70000]
})

print(df)
#      name  age  salary
```

```
# 0   Alice   25   50000
# 1     Bob   30   60000
# 2 Charlie   35   70000

# Each column is a Series
print(type(df['name']))  # <class 'pandas.core.series.Series'>
```

3. Index -- The Labels

The Index is the "row labels" of a DataFrame. It enables:

- Fast lookups
- Data alignment
- Hierarchical (multi-level) indexing

```
df = pd.DataFrame(
    {'A': [1, 2, 3], 'B': [4, 5, 6]},
    index=['row1', 'row2', 'row3']
)
print(df.index)  # Index(['row1', 'row2', 'row3'], dtype='object')
```

Creating DataFrames: All Methods

```
import pandas as pd

# 1. From dictionary (most common)
df1 = pd.DataFrame({
    'col1': [1, 2, 3],
    'col2': ['a', 'b', 'c']
})

# 2. From list of lists (with columns)
df2 = pd.DataFrame(
    [[1, 'a'], [2, 'b'], [3, 'c']],
    columns=['col1', 'col2']
)

# 3. From list of dictionaries
df3 = pd.DataFrame([
    {'name': 'Alice', 'age': 25},
    {'name': 'Bob', 'age': 30}
])

# 4. From numpy array
import numpy as np
df4 = pd.DataFrame(
    np.random.randn(4, 3),
    columns=['A', 'B', 'C']
)

# 5. From CSV file (we'll cover in Module 01)
# df5 = pd.read_csv('data.csv')

# 6. From Excel file
# df6 = pd.read_excel('data.xlsx')
```

Essential DataFrame Attributes

```
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'age': [25, 30, 35, 28],
    'salary': [50000, 60000, 70000, 55000],
    'department': ['Engineering', 'Sales', 'Engineering', 'Marketing']
})

# Shape: (rows, columns)
print(df.shape)  # (4, 4)

# Columns: column names
print(df.columns)  # Index(['name', 'age', 'salary', 'department'], dtype='object')

# Index: row labels
print(df.index)  # RangeIndex(start=0, stop=4, step=1)

# dtypes: data type of each column
print(df.dtypes)
# name         object
# age           int64
# salary        int64
# department   object
# dtype: object

# Values: underlying numpy array
print(df.values)

# T: transpose (swap rows and columns)
print(df.T)

# info(): memory and type summary
print(df.info())

# memory_usage(): how much RAM each column uses
print(df.memory_usage())
```

Setting and Resetting Index

```
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'salary': [50000, 60000, 70000]
})

# Set a column as index (why? faster lookups, meaningful labels)
df_indexed = df.set_index('name')
print(df_indexed)
#          age  salary
# name
# Alice     25   50000
# Bob       30   60000
# Charlie   35   70000

# Reset index (convert index back to column)
df_reset = df_indexed.reset_index()
print(df_reset)
```

```
# Why set an index?
# 1. Faster lookups by label
# 2. Meaningful row identifiers
# 3. Required for time-series operations
# 4. Enables hierarchical indexing
```

Viewing Data

```
# First N rows
df.head(3)

# Last N rows
df.tail(2)

# Random sample
df.sample(3)            # 3 random rows
df.sample(frac=0.5)     # 50% of rows randomly

# Specific rows by position
df.iloc[0]              # First row
df.iloc[0:3]            # First 3 rows
df.iloc[[0, 2]]         # Rows 0 and 2

# Specific rows by label
df.loc[0]               # Row with index label 0
df.loc[0:2]             # Rows with labels 0, 1, 2 (inclusive!)
```

Important: <code>loc</code> is label-based (inclusive of end), <code>iloc</code> is position-based (exclusive of end).

Next Steps

- **Module 01:** Learn to load data from CSV, JSON, Excel, SQL, APIs
- **Module 02:** Explore and understand your data (EDA)
- **Module 03:** Clean and prepare messy real-world data

Quick Reference

Operation	Code	Description
Create DataFrame	`pd.DataFrame(dict)`	From dictionary
View shape	`df.shape`	(rows, cols)
View columns	`df.columns`	Column names

View types	`df.dtypes`	Data types
First rows	`df.head(n)`	Top n rows
Random sample	`df.sample(n)`	Random n rows
Set index	`df.set_index('col')`	Use column as index
Reset index	`df.reset_index()`	Convert index to column
Info	`df.info()`	Structure summary
Memory	`df.memory_usage()`	RAM usage

MODULE-01a: DATA INGESTION -- CSV AND JSON

The Data Loading Landscape

Real-world data comes in many formats. Pandas provides <code>read_*</code> functions for each:

Function	Format	Best For
`read_csv()`	CSV/TSV	Flat files, exports from systems
`read_json()`	JSON	APIs, NoSQL databases
`read_excel()`	Excel (.xlsx)	Business reports, spreadsheets
`read_sql()`	SQL databases	Relational databases
`read_parquet()`	Parquet	Big data, analytics pipelines

---

1. CSV FILES -- The Workhorse

Basic Loading

```
import pandas as pd

# Minimal -- just the file path
df = pd.read_csv('data.csv')

# With TSV (tab-separated)
df = pd.read_csv('data.tsv', sep='\t')
```

Critical Parameters (and WHY they matter)

```
# --- HEADER & ROWS ---
df = pd.read_csv('data.csv',
    header=0,           # Row to use as column names (0 = first row)
                        # Why? Some files have metadata rows at top
    index_col=0,        # Column to use as row index
                        # Why? Faster lookups, meaningful labels
    skiprows=[1, 3],    # Skip specific row numbers (0-indexed)
    skipfooter=2,       # Skip rows at the end
    nrows=100,          # Only read first 100 rows
                        # Why? Preview large files without loading all
)

# --- DATA TYPES ---
df = pd.read_csv('data.csv',
    dtype={'age': 'int32', 'salary': 'float64'},
                        # Force specific types
                        # Why? Save memory, prevent type errors
    parse_dates=['created_date', 'updated_date'],
                        # Parse as datetime instead of string
```

```
                    # Why? Enable time-series operations
)

# --- MISSING VALUES ---
df = pd.read_csv('data.csv',
    na_values=['NA', 'N/A', '-', 'NULL', ''],
                    # Treat these strings as NaN
                    # Why? Different systems use different null markers
    keep_default_na=True,
)

# --- ENCODING ---
df = pd.read_csv('data.csv',
    encoding='utf-8',     # Standard encoding
    encoding='latin-1',   # For European characters
                    # Why? Wrong encoding = garbled text
)

# --- CHUNKING (Large Files) ---
# Why chunk? Files larger than RAM will crash your program
chunk_size = 100_000
chunks = []
for chunk in pd.read_csv('large_file.csv', chunksize=chunk_size):
    filtered = chunk[chunk['status'] == 'active']
    chunks.append(filtered)
df = pd.concat(chunks, ignore_index=True)

# --- USECOLS (Memory Optimization) ---
df = pd.read_csv('data.csv', usecols=['name', 'age', 'salary'])
                    # Why? Faster loading, less memory
```

Common CSV Pitfalls

```
# Problem: Comma in quoted fields
# Solution: Pandas handles this automatically with quotechar='"'

# Problem: Mixed line endings (Windows \r\n vs Unix \n)
# Solution: Use lineterminator='\n' or let pandas auto-detect

# Problem: Inconsistent number of fields
# Solution: on_bad_lines parameter
df = pd.read_csv('data.csv', on_bad_lines='warn')
                    # Options: 'error', 'warn', 'skip'
```

---

2. JSON FILES

JSON (JavaScript Object Notation) is hierarchical. Pandas needs help flattening it.

Simple JSON (List of Dicts)

```
# JSON array of objects → DataFrame directly
df = pd.read_json('data.json')

# From JSON string
import json
```

```
json_str = '[{"name": "Alice", "age": 25}, {"name": "Bob", "age": 30}]'
df = pd.read_json(json_str)
```

Nested JSON -- The Real Challenge

```
import json
from pandas import json_normalize

# Nested structure (common from APIs)
nested_data = {
    "users": [
        {
            "id": 1,
            "name": "Alice",
            "address": {"city": "NYC", "zip": "10001"},
            "orders": [{"id": 101, "amount": 50}, {"id": 102, "amount": 75}]
        },
        {
            "id": 2,
            "name": "Bob",
            "address": {"city": "LA", "zip": "90001"},
            "orders": [{"id": 103, "amount": 100}]
        }
    ]
}

# Method 1: json_normalize (flatten nested structures)
# Flatten one level
df = json_normalize(nested_data['users'])
print(df)
#    id   name address.city address.zip  ...
# 0   1  Alice          NYC       10001  ...
# 1   2    Bob           LA       90001  ...

# Flatten with record_path (extract nested lists)
df_orders = json_normalize(
    nested_data['users'],
    record_path='orders',          # The nested list to extract
    meta=['id', 'name',            # Columns to keep from parent
          ['address', 'city']]     # Nested parent columns
)
print(df_orders)
#    id  amount name address.city
# 0 101      50  Alice          NYC
# 1 102      75  Alice          NYC
# 2 103     100    Bob           LA
```

JSON Orientation

```
# Pandas can read JSON in different orientations
# 'records': [{"col": val}, ...]  (default, most common)
# 'split':   {"index": [...], "columns": [...], "data": [[...]]}
# 'index':   {row_idx: {col: val}}
# 'columns': {col: {row_idx: val}}
# 'values':  [[val, val], ...]

# Example: reading 'split' orientation
df = pd.read_json('data.json', orient='split')
```

Quick Reference

Task	Code
Load CSV	`pd.read_csv('file.csv')`
Load TSV	`pd.read_csv('file.tsv', sep='\t')`
Load JSON	`pd.read_json('file.json')`
Preview large file	`pd.read_csv('file.csv', nrows=100)`
Load specific columns	`pd.read_csv('file.csv', usecols=['a', 'b'])`
Parse dates	`pd.read_csv('file.csv', parse_dates=['date'])`
Handle missing values	`pd.read_csv('file.csv', na_values=['NA', '-'])`
Chunk large file	`pd.read_csv('file.csv', chunksize=10000)`
Flatten nested JSON	`json_normalize(data, record_path='items')`

Next Steps

- **Module 01b:** Excel, SQL, API, and Parquet loading
- **Module 02a:** Basic data exploration (EDA)

MODULE-01b: DATA INGESTION -- EXCEL, SQL, APIs, AND MORE

---

3. EXCEL FILES

Excel files can contain multiple sheets, formatting, and formulas.

Basic Loading

```
import pandas as pd

# Single sheet
df = pd.read_excel('data.xlsx')              # First sheet by default
df = pd.read_excel('data.xlsx', sheet_name='Sheet2')
df = pd.read_excel('data.xlsx', sheet_name=0) # By index (0-based)

# Multiple sheets at once
sheets = pd.read_excel('data.xlsx', sheet_name=None)
                    # Returns dict: {sheet_name: DataFrame}
for name, df in sheets.items():
    print(f"Sheet: {name}, Shape: {df.shape}")
```

Important Parameters

```
df = pd.read_excel('data.xlsx',
    sheet_name='Data',
    header=0,               # Row for column names
    index_col=0,            # Column for row index
    usecols='A:D',          # Load specific columns (Excel notation)
    usecols=[0, 2, 3],      # Or by position
    skiprows=2,             # Skip header rows
    nrows=1000,             # Limit rows
    dtype={'ID': str},      # Force types
    parse_dates=['Date'],   # Parse date columns
    na_values=['NA', 'N/A', '-'],
    engine='openpyxl',      # For .xlsx files
    engine='xlrd',          # For .xls files (older format)
)
```

Multi-Header Excel Files

```
# Some Excel files have multi-row headers
df = pd.read_excel('data.xlsx', header=[0, 1])
                    # Creates MultiIndex columns
# Access: df[('Header1', 'SubHeader1')]
```

---

4. SQL DATABASES

```
from sqlalchemy import create_engine

# Create engine (supports PostgreSQL, MySQL, SQLite, etc.)
engine = create_engine('sqlite:///mydatabase.db')
engine = create_engine('postgresql://user:pass@host:5432/dbname')
engine = create_engine('mysql+pymysql://user:pass@host:3306/dbname')

# Read SQL query
df = pd.read_sql("SELECT * FROM users WHERE active = 1", engine)

# Read entire table
df = pd.read_sql_table('users', engine)

# Read with parameters (prevents SQL injection)
df = pd.read_sql(
    "SELECT * FROM users WHERE department = %s",
    engine,
    params=['Engineering']
)

# Chunk large results
chunk_iter = pd.read_sql("SELECT * FROM large_table", engine, chunksize=10000)
for chunk in chunk_iter:
    process(chunk)
```

---

5. API DATA (JSON from REST APIs)

```
import requests
from pandas import json_normalize

# Fetch data from API
response = requests.get('https://api.example.com/users')
response.raise_for_status()  # Raise error for bad responses
data = response.json()

# Convert to DataFrame
df = pd.DataFrame(data['results'])  # Adjust based on API structure

# Handle nested API responses
df = json_normalize(
    data['results'],
    record_path='items',
    meta=['page', 'total_count']
)
```

---

6. OTHER FORMATS

```
# Parquet (columnar, compressed -- best for big data)
df = pd.read_parquet('data.parquet')
```

```
# Feather (fast binary format)
df = pd.read_feather('data.feather')

# Pickle (Python-specific, fast but not portable)
df = pd.read_pickle('data.pkl')

# HTML tables (web scraping)
tables = pd.read_html('https://example.com/table-page.html')
                    # Returns list of DataFrames
df = tables[0]

# Clipboard (quick copy-paste)
df = pd.read_clipboard()  # Copy from Excel, paste directly
```

Production-Grade Loading Pattern

```
import pandas as pd
import logging
from pathlib import Path

def load_data_safely(file_path: str, **kwargs) -> pd.DataFrame:
    """
    Production-grade data loader with error handling.

    Args:
        file_path: Path to data file
        **kwargs: Additional arguments passed to read_* functions

    Returns:
        DataFrame with loaded data
    """
    path = Path(file_path)

    if not path.exists():
        raise FileNotFoundError(f"File not found: {file_path}")

    # Determine format from extension
    ext = path.suffix.lower()
    loaders = {
        '.csv': pd.read_csv,
        '.tsv': lambda p: pd.read_csv(p, sep='\t'),
        '.json': pd.read_json,
        '.xlsx': pd.read_excel,
        '.xls': pd.read_excel,
        '.parquet': pd.read_parquet,
    }

    if ext not in loaders:
        raise ValueError(f"Unsupported format: {ext}")

    try:
        df = loaders[ext](file_path, **kwargs)
        logging.info(f"Loaded {len(df)} rows, {len(df.columns)} columns from {file_path}")
        return df
    except Exception as e:
        logging.error(f"Failed to load {file_path}: {e}")
        raise

# Usage
df = load_data_safely('data/sales_2024.csv', parse_dates=['date'])
```

Quick Reference

Task	Code
Load Excel	`pd.read_excel('file.xlsx')`
Multiple sheets	`pd.read_excel('file.xlsx', sheet_name=None)`
Load SQL	`pd.read_sql('SELECT...', engine)`
Load from API	`pd.DataFrame(requests.get(url).json())`
Load Parquet	`pd.read_parquet('file.parquet')`
Load HTML table	`pd.read_html(url)`
Load from clipboard	`pd.read_clipboard()`

Next Steps

- **Module 02a:** Basic data exploration (EDA)
- **Module 07a:** Export to CSV and Excel

MODULE-02a: DATA EXPLORATION -- BASIC EDA

What is EDA and Why It Matters

Exploratory Data Analysis (EDA) is the process of understanding your data before modeling or reporting.

Why EDA first?

- Reveals data quality issues (missing values, outliers, errors)
- Uncovers patterns and relationships
- Informs feature engineering decisions
- Prevents garbage-in-garbage-out scenarios

> *"The greatest value of a picture is when it forces us to notice what we never expected to see." -- John Tukey*

Loading Sample Data

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'employee_id': range(1, 101),
    'name': [f'Employee_{i}' for i in range(1, 101)],
    'department': np.random.choice(['Engineering', 'Sales', 'Marketing', 'HR', 'Finance'], 100),
    'age': np.random.randint(22, 65, 100),
    'salary': np.random.randint(40000, 150000, 100),
    'years_experience': np.random.randint(0, 40, 100),
    'performance_score': np.round(np.random.uniform(1.0, 5.0, 100), 1),
    'hire_date': pd.date_range('2015-01-01', periods=100, freq='3D'),
    'remote_work': np.random.choice([True, False], 100),
    'city': np.random.choice(['New York', 'London', 'Tokyo', 'Sydney', 'Berlin'], 100)
})

# Add some missing values for realism
df.loc[df.sample(10).index, 'salary'] = np.nan
df.loc[df.sample(5).index, 'performance_score'] = np.nan
```

Step 1: First Look -- Shape and Structure

```
# --- DIMENSIONS ---
print(f"Shape: {df.shape}")          # (rows, columns)
print(f"Rows: {df.shape[0]}")        # Number of records
print(f"Columns: {df.shape[1]}")     # Number of features

# --- COLUMN NAMES ---
```

```
print(df.columns.tolist())          # List of column names
print(df.columns)                   # Index object

# --- DATA TYPES ---
print(df.dtypes)
# Why check dtypes?
# - object columns might need conversion to category
# - numeric columns stored as strings need conversion
# - datetime columns need parsing

# --- DETAILED STRUCTURE ---
print(df.info())
# Shows: column names, non-null counts, dtypes, memory usage
# Why? Quick scan for missing data and type issues
```

Step 2: Sample the Data

```
# --- FIRST ROWS ---
print(df.head())                    # First 5 rows (default)
print(df.head(10))                  # First 10 rows

# --- LAST ROWS ---
print(df.tail())                    # Last 5 rows
# Why check tail? Sometimes data quality issues appear at end

# --- RANDOM SAMPLES ---
print(df.sample(5))                 # 5 random rows
print(df.sample(n=5, random_state=42))  # Reproducible sample
print(df.sample(frac=0.1))          # 10% of data

# --- SPECIFIC ROWS ---
print(df.iloc[0])                   # First row (by position)
print(df.loc[0])                    # Row with index label 0
```

Step 3: Numerical Summary Statistics

```
# --- BASIC DESCRIPTION ---
print(df.describe())
# Shows: count, mean, std, min, 25%, 50%, 75%, max
# Why? Quick overview of distribution and potential outliers

# --- INCLUDE ALL COLUMNS ---
print(df.describe(include='all'))
# For object columns: shows count, unique, top (most frequent), freq

# --- SPECIFIC STATISTICS ---
print(df['salary'].mean())          # Average
print(df['salary'].median())        # Middle value (robust to outliers)
print(df['salary'].mode())          # Most frequent value
print(df['salary'].std())           # Standard deviation (spread)
print(df['salary'].min())           # Minimum
print(df['salary'].max())           # Maximum
print(df['salary'].quantile(0.90))  # 90th percentile
print(df['salary'].quantile([0.25, 0.50, 0.75]))  # Quartiles
```

```
# --- WHY MEAN VS MEDIAN? ---
# Mean is sensitive to outliers; median is robust
# Use median when data is skewed (income, house prices, etc.)
```

Step 4: Categorical Data Analysis

```
# --- VALUE COUNTS ---
print(df['department'].value_counts())
# Why? Understand class distribution, spot imbalances

# --- WITH PERCENTAGES ---
print(df['department'].value_counts(normalize=True) * 100)
# Shows percentage of each category

# --- SORTED BY FREQUENCY ---
print(df['department'].value_counts().sort_index())
# Sort alphabetically instead of by frequency

# --- BINARY COLUMNS ---
print(df['remote_work'].value_counts())
# Quick check of True/False distribution

# --- UNIQUE VALUES ---
print(df['department'].nunique())    # Number of unique values
print(df['department'].unique())     # List of unique values

# --- CROSS-TABULATION ---
print(pd.crosstab(df['department'], df['remote_work']))
# Why? See relationships between two categorical variables
```

Step 5: Missing Data Analysis

```
# --- COUNT MISSING VALUES ---
print(df.isnull().sum())            # Missing values per column
print(df.isnull().sum().sum())      # Total missing values

# --- PERCENTAGE MISSING ---
missing_pct = df.isnull().mean() * 100
print(missing_pct[missing_pct > 0])  # Only columns with missing data

# --- ROWS WITH ANY MISSING ---
print(df[df.isnull().any(axis=1)])   # Show rows with any NaN

# --- PATTERNS OF MISSINGNESS ---
# Why analyze patterns?
# - MCAR (Missing Completely At Random): no pattern
# - MAR (Missing At Random): depends on observed data
# - MNAR (Missing Not At Random): depends on unobserved data
```

Step 6: Quick Visual EDA

```
import matplotlib.pyplot as plt

# --- HISTOGRAM ---
df['salary'].hist(bins=30, edgecolor='black')
plt.title('Salary Distribution')
plt.xlabel('Salary')
plt.ylabel('Count')
plt.savefig('charts/eda_salary_hist.png', dpi=150, bbox_inches='tight')
plt.close()

# --- BOX PLOT BY CATEGORY ---
df.boxplot(column='salary', by='department', figsize=(10, 6))
plt.title('Salary by Department')
plt.suptitle('')
plt.ylabel('Salary')
plt.savefig('charts/eda_salary_by_dept.png', dpi=150, bbox_inches='tight')
plt.close()

# --- SCATTER PLOT ---
plt.scatter(df['years_experience'], df['salary'], alpha=0.6)
plt.title('Experience vs Salary')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.savefig('charts/eda_experience_vs_salary.png', dpi=150, bbox_inches='tight')
plt.close()
```

Quick Reference

Task	Code
Shape	`df.shape`
First rows	`df.head(n)`
Random sample	`df.sample(n)`
Column types	`df.dtypes`
Structure info	`df.info()`
Numerical summary	`df.describe()`
All columns summary	`df.describe(include='all')`
Value counts	`df['col'].value_counts()`
Missing count	`df.isnull().sum()`
Missing %	`df.isnull().mean() * 100`
Duplicates	`df.duplicated().sum()`

Next Steps

- **Module 02b:** Advanced EDA (correlation, distributions, EDA function)
- **Module 03a:** Missing values -- detection and imputation

MODULE-02b: DATA EXPLORATION -- ADVANCED EDA

Step 7: Correlation Analysis

```
import numpy as np

# --- NUMERICAL CORRELATION ---
correlation = df.select_dtypes(include=[np.number]).corr()
print(correlation)
# Why? Identify relationships between numerical variables
# Values: -1 (perfect negative) to +1 (perfect positive)

# --- CORRELATION WITH TARGET ---
# If you have a target variable:
print(correlation['salary'].sort_values(ascending=False))
# Which features correlate most with salary?

# --- STRONG CORRELATIONS ---
# Find highly correlated pairs (potential multicollinearity)
def find_high_correlations(df, threshold=0.7):
    corr = df.corr()
    upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
    return upper.stack().sort_values(ascending=False)

print(find_high_correlations(df, threshold=0.3))

# --- SPEARMAN CORRELATION ---
# For non-linear relationships or ordinal data
print(df.select_dtypes(include=[np.number]).corr(method='spearman'))
```

Step 8: Distribution Analysis

```
# --- HISTOGRAM DATA ---
print(df['salary'].value_counts(bins=10).sort_index())
# Bin numerical data into 10 ranges

# --- SKEWNESS ---
print(df['salary'].skew())
# Why? Skewness tells you if distribution is asymmetric
# Positive skew: tail on right (most values low, few very high)
# Negative skew: tail on left (most values high, few very low)
# ~0: symmetric distribution

# --- KURTOSIS ---
print(df['salary'].kurt())
# Why? Kurtosis tells you about tail heaviness
# High kurtosis: heavy tails (more outliers)
# Low kurtosis: light tails (fewer outliers)

# --- GROUPED STATISTICS ---
```

```
print(df.groupby('department')['salary'].agg(['mean', 'median', 'std', 'count']))
# Why? Compare distributions across categories
```

Step 9: Correlation Heatmap

```
import matplotlib.pyplot as plt
import seaborn as sns

# --- CORRELATION HEATMAP ---
plt.figure(figsize=(10, 8))
sns.heatmap(df.select_dtypes(include=[np.number]).corr(), annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Matrix')
plt.savefig('charts/eda_correlation_heatmap.png', dpi=150, bbox_inches='tight')
plt.close()
```

Step 10: Production EDA Function

```
def quick_eda(df, target_col=None):
    """
    Production-grade quick EDA function.
    Run this on any new dataset.
    """
    print("=" * 60)
    print("QUICK EDA REPORT")
    print("=" * 60)

    # 1. Shape
    print(f"\n Shape: {df.shape[0]} rows × {df.shape[1]} columns")

    # 2. Types
    print(f"\n Data Types:")
    print(df.dtypes.value_counts())

    # 3. Missing data
    missing = df.isnull().sum()
    missing_pct = (missing / len(df)) * 100
    print(f"\n Missing Data:")
    for col in df.columns:
        if missing[col] > 0:
            print(f"  {col}: {missing[col]} ({missing_pct[col]:.1f}%)")

    # 4. Duplicates
    dupes = df.duplicated().sum()
    print(f"\n Duplicates: {dupes} ({dupes/len(df)*100:.1f}%)")

    # 5. Numerical summary
    print(f"\n Numerical Columns:")
    print(df.select_dtypes(include=[np.number]).describe())

    # 6. Categorical summary
    print(f"\n Categorical Columns:")
    cat_cols = df.select_dtypes(include=['object', 'category']).columns
    for col in cat_cols:
        print(f"\n  {col}:")
        print(f"    Unique: {df[col].nunique()}")
```

```
        print(f"    Top 3: {df[col].value_counts().head(3).to_dict()}")

    # 7. Correlations (if target specified)
    if target_col:
        print(f"\n Correlations with '{target_col}':")
        num_cols = df.select_dtypes(include=[np.number]).columns
        corr = df[num_cols].corr()[target_col].sort_values(ascending=False)
        print(corr)

    print("\n" + "=" * 60)

# Usage
quick_eda(df, target_col='salary')
```

Quick Reference

Task	Code
Correlation	`df.corr()`
Skewness	`df['col'].skew()`
Grouped stats	`df.groupby('col')['val'].agg(['mean', 'std'])`
Heatmap	`sns.heatmap(df.corr(), annot=True)`
Full EDA	`quick_eda(df, target_col='salary')`

Next Steps

- **Module 03a:** Missing values -- detection and imputation
- **Module 06a:** Basic visualization (line, bar, scatter)

MODULE-03a: DATA CLEANING -- MISSING VALUES

Why Data Cleaning Matters

Real-world data is messy. Studies show data scientists spend 60-80% of their time on data cleaning.

---

1. Missing Values -- Detection and Analysis

```
import pandas as pd
import numpy as np

# Create messy dataset
df = pd.DataFrame({
    'id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'name': ['Alice', 'Bob', None, 'Diana', 'Eve', 'Frank', '', 'Hannah', 'Ivan', 'Jane'],
    'age': [25, 30, np.nan, 35, 28, np.nan, 45, 32, np.nan, 40],
    'salary': [50000, 60000, 55000, np.nan, 70000, 65000, np.nan, 58000, 72000, np.nan],
    'department': ['Eng', 'Sales', 'Eng', 'HR', 'Sales', 'Eng', 'HR', 'Sales', 'Eng', 'HR'],
    'email': ['alice@co.com', 'bob@co.com', 'dan@co.com', None, 'eve@co.com',
              'frank@co.com', 'george@co.com', 'hannah@co.com', 'ivan@co.com', 'jane@co.com']
})

# --- DETECT MISSING VALUES ---
print(df.isnull())          # Boolean DataFrame (True where missing)
# Why isnull() and isna()? They're identical aliases (pandas supports both)

# --- COUNT MISSING ---
print(df.isnull().sum())          # Missing per column
print(df.isnull().sum().sum())    # Total missing

# --- PERCENTAGE MISSING ---
missing_pct = df.isnull().mean() * 100
print(missing_pct)

# --- ROWS WITH ANY MISSING ---
print(df[df.isnull().any(axis=1)])

# --- NON-MISSING COUNT ---
print(df.notnull().sum())         # Non-missing per column
```

---

2. Missing Values -- Strategies and Imputation

Strategy Decision Tree

```
Missing Data?
├── <5% missing → Drop rows (if MCAR)
├── 5-20% missing → Impute (mean/median/mode)
```

```
├── 20-50% missing → Advanced imputation or flag as separate category
└── >50% missing → Consider dropping column
```

Drop Missing Values

```
# --- DROP ROWS ---
df_dropped = df.dropna()
                    # Drop rows with ANY missing value
df_dropped = df.dropna(subset=['salary'])
                    # Drop rows where 'salary' is missing
df_dropped = df.dropna(how='all')
                    # Drop only if ALL values are missing
df_dropped = df.dropna(thresh=3)
                    # Keep rows with at least 3 non-missing values

# --- DROP COLUMNS ---
df_dropped_cols = df.dropna(axis=1)
                    # Drop columns with any missing
df_dropped_cols = df.dropna(axis=1, thresh=int(0.8 * len(df)))
                    # Drop columns with >20% missing

# Why drop vs impute?
# Drop: Simple, but loses data. Only use when missing is minimal and random.
# Impute: Preserves data, but introduces assumptions.
```

Simple Imputation

```
# --- NUMERICAL COLUMNS ---
# Mean imputation (use when data is normally distributed)
df['age'] = df['age'].fillna(df['age'].mean())

# Median imputation (use when data is skewed -- robust to outliers)
df['salary'] = df['salary'].fillna(df['salary'].median())

# Why mean vs median?
# Mean: sensitive to outliers. Use for symmetric distributions.
# Median: robust to outliers. Use for skewed distributions (income, prices).

# --- CATEGORICAL COLUMNS ---
# Mode imputation (most frequent value)
df['department'] = df['department'].fillna(df['department'].mode()[0])

# --- STRING COLUMNS ---
df['name'] = df['name'].fillna('Unknown')
df['email'] = df['email'].fillna('no-email@company.com')

# --- FORWARD / BACKWARD FILL ---
# For time-series data (fill with previous/next value)
df['salary'] = df['salary'].ffill()  # Forward fill
df['salary'] = df['salary'].bfill()  # Backward fill
# Why? In time series, missing values are often similar to adjacent values
```

Advanced Imputation

```
# --- GROUP-BASED IMPUTATION ---
# Fill missing salary with department median
df['salary'] = df.groupby('department')['salary'].transform(
```

```
    lambda x: x.fillna(x.median())
)
# Why group-based? Different departments have different salary ranges

# --- INTERPOLATION ---
# Linear interpolation (estimates missing values between known values)
df['age'] = df['age'].interpolate(method='linear')
# Why interpolate? Better than mean for ordered/time-series data

# --- KNN IMPUTATION ---
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=3)
numeric_cols = df.select_dtypes(include=[np.number]).columns
df[numeric_cols] = imputer.fit_transform(df[numeric_cols])
# Why KNN? Uses similar rows to estimate missing values
```

Quick Reference

Task	Code
Count missing	`df.isnull().sum()`
Drop rows with NaN	`df.dropna()`
Fill with mean	`df['col'].fillna(df['col'].mean())`
Fill with median	`df['col'].fillna(df['col'].median())`
Forward fill	`df['col'].ffill()`
Group-based fill	`df.groupby('g')['c'].transform(lambda x: x.fillna(x.median()))`

Next Steps

- **Module 03b:** Duplicates, type conversion, string cleaning, outliers
- **Module 04a:** Indexing and filtering

MODULE-03b: DATA CLEANING -- DUPLICATES, TYPES, STRINGS, OUTLIERS

---

3. Duplicate Detection and Removal

```
# --- DETECT DUPLICATES ---
print(df.duplicated())             # Boolean series (True if duplicate)
print(df.duplicated().sum())       # Count of duplicate rows
print(df[df.duplicated(keep=False)])  # Show all duplicates (including first occurrence)

# --- REMOVE DUPLICATES ---
df_clean = df.drop_duplicates()
                    # Remove exact duplicate rows
df_clean = df.drop_duplicates(subset=['email'])
                    # Remove duplicates based on specific column
df_clean = df.drop_duplicates(subset=['name', 'email'], keep='last')
                    # Keep last occurrence

# Why keep='first' vs 'last'?
# 'first': Keep original record, remove later entries
# 'last': Keep most recent record (useful for updated data)
```

---

4. Type Conversion and Validation

```
# --- CHECK CURRENT TYPES ---
print(df.dtypes)

# --- CONVERT TYPES ---
# String to numeric
df['age'] = pd.to_numeric(df['age'], errors='coerce')
                    # errors='coerce': invalid values become NaN
                    # errors='raise': raise exception on invalid
                    # errors='ignore': leave invalid values as-is

# String to datetime
df['hire_date'] = pd.to_datetime(df['hire_date'], format='%Y-%m-%d')

# Numeric to categorical
df['department'] = df['department'].astype('category')
                    # Why category? Saves memory, faster operations
                    # Use when: few unique values relative to row count

# Integer with NaN (pandas nullable integer)
df['age'] = df['age'].astype('Int64')  # Capital I -- nullable integer
                    # Why? Standard int64 cannot hold NaN

# --- VALIDATE TYPES ---
def validate_types(df, expected_types):
    """Validate that columns have expected types."""
    issues = []
    for col, expected in expected_types.items():
```

```
        actual = str(df[col].dtype)
        if actual != expected:
            issues.append(f"  {col}: expected {expected}, got {actual}")
    if issues:
        print("Type validation issues:")
        for issue in issues:
            print(issue)
    else:
        print("All types valid ✓")

validate_types(df, {
    'age': 'int64',
    'salary': 'float64',
    'department': 'category'
})
```

---

5. String Cleaning

```
# --- COMMON STRING ISSUES ---
df['name'] = df['name'].str.strip()         # Remove leading/trailing whitespace
df['name'] = df['name'].str.lower()         # Convert to lowercase
df['name'] = df['name'].str.upper()         # Convert to uppercase
df['name'] = df['name'].str.title()         # Title case (First Letter Capital)
df['name'] = df['name'].str.replace('  ', ' ')  # Replace double spaces

# --- EXTRACTION ---
# Extract parts of strings
df['domain'] = df['email'].str.extract(r'@(.+)$')
                    # Extract domain from email
df['first_name'] = df['name'].str.split().str[0]
                    # Extract first word
df['last_name'] = df['name'].str.split().str[-1]
                    # Extract last word

# --- PATTERN MATCHING ---
print(df['email'].str.contains(r'@.+\..+'))  # Basic email pattern
print(df['email'].str.match(r'^[a-z]+@'))     # Starts with lowercase letters

# --- REPLACEMENT ---
df['department'] = df['department'].str.replace('Eng', 'Engineering')
df['department'] = df['department'].replace({
    'Eng': 'Engineering',
    'Sales': 'Sales',
    'HR': 'Human Resources'
})

# --- LENGTH AND COUNT ---
print(df['email'].str.len())                # String length
print(df['email'].str.count('@'))           # Count occurrences of @
```

---

6. Outlier Detection and Handling

```
# --- STATISTICAL OUTLIER DETECTION ---
# IQR Method (Interquartile Range)
Q1 = df['salary'].quantile(0.25)
```

```
Q3 = df['salary'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df['salary'] < lower_bound) | (df['salary'] > upper_bound)]
print(f"Outliers detected: {len(outliers)}")

# Z-Score Method
from scipy import stats
z_scores = np.abs(stats.zscore(df['salary'].dropna()))
outliers_z = df['salary'].dropna()[z_scores > 3]
print(f"Z-score outliers (>3σ): {len(outliers_z)}")

# --- HANDLE OUTLIERS ---
# Option 1: Cap values (winsorize)
df['salary_capped'] = df['salary'].clip(lower=lower_bound, upper=upper_bound)
# Why cap? Preserves data but limits extreme influence

# Option 2: Remove outliers
df_no_outliers = df[(df['salary'] >= lower_bound) & (df['salary'] <= upper_bound)]

# Option 3: Log transform (for right-skewed data)
df['salary_log'] = np.log1p(df['salary'])
# Why log? Compresses large values, expands small values
# log1p = log(1+x) -- handles zeros safely
```

---

7. Data Validation Patterns

```
# --- BUSINESS RULE VALIDATION ---
def validate_data(df):
    """Production-grade data validation."""
    issues = []

    # Check for negative ages
    if (df['age'] < 0).any():
        issues.append("Negative ages found")

    # Check for future dates
    if (df['hire_date'] > pd.Timestamp.now()).any():
        issues.append("Future hire dates found")

    # Check for impossible salaries
    if (df['salary'] < 0).any():
        issues.append("Negative salaries found")

    # Check for empty strings in required fields
    if df['name'].str.strip().eq('').any():
        issues.append("Empty names found")

    # Check for duplicate IDs
    if df['id'].duplicated().any():
        issues.append("Duplicate IDs found")

    return issues

issues = validate_data(df)
if issues:
    print("Validation issues:")
    for issue in issues:
```

```
        print(f"  ⚠ {issue}")
else:
    print("✓ All validations passed")
```

Quick Reference

Task	Code
Remove duplicates	`df.drop_duplicates()`
Convert type	`df['col'].astype('float')`
String to numeric	`pd.to_numeric(df['col'], errors='coerce')`
String to date	`pd.to_datetime(df['col'])`
Strip whitespace	`df['col'].str.strip()`
Detect outliers (IQR)	`Q1 - 1.5*IQR` to `Q3 + 1.5*IQR`
Cap outliers	`df['col'].clip(lower, upper)`

Next Steps

- **Module 04a:** Indexing and filtering
- **Module 06a:** Basic visualization

MODULE-04a: DATA MANIPULATION -- INDEXING, FILTERING, SORTING

---

1. Indexing and Selection

The Four Accessors

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'age': [25, 30, 35, 28, 32],
    'salary': [50000, 60000, 70000, 55000, 65000],
    'department': ['Eng', 'Sales', 'Eng', 'HR', 'Sales']
}, index=['a', 'b', 'c', 'd', 'e'])

# --- loc: LABEL-based ---
# Syntax: df.loc[row_labels, column_labels]
print(df.loc['a'])                  # Row 'a' (all columns)
print(df.loc['a', 'name'])          # Single value
print(df.loc['a':'c'])              # Slice (inclusive of 'c'!)
print(df.loc[['a', 'c'], ['name', 'salary']])  # Specific rows and columns
print(df.loc[df['age'] > 28])       # Boolean filter by label

# --- iloc: POSITION-based ---
# Syntax: df.iloc[row_positions, column_positions]
print(df.iloc[0])                   # First row
print(df.iloc[0, 0])                # First value (row 0, col 0)
print(df.iloc[0:3])                 # First 3 rows (exclusive of 3)
print(df.iloc[[0, 2], [0, 2]])      # Rows 0,2 and cols 0,2
print(df.iloc[:, 1:3])              # All rows, columns 1-2

# --- at: LABEL-based (single value, faster) ---
print(df.at['a', 'name'])           # Faster than loc for single value

# --- iat: POSITION-based (single value, faster) ---
print(df.iat[0, 0])                 # Faster than iloc for single value

# WHY FOUR ACCESSORS?
# loc/iloc: Flexible, support slices and lists
# at/iat: Fast, single value only (optimization)
```

Common Indexing Patterns

```
# --- SELECT SINGLE COLUMN ---
df['name']          # Returns Series
df[['name']]        # Returns DataFrame (note double brackets)
df.name             # Attribute access (only works if valid Python identifier)

# --- SELECT MULTIPLE COLUMNS ---
df[['name', 'salary']]
```

```
# --- SELECT ROWS BY CONDITION ---
df[df['age'] > 28]
df[(df['age'] > 28) & (df['department'] == 'Eng')]  # Multiple conditions
df[(df['age'] > 28) | (df['department'] == 'HR')]   # OR condition

# --- QUERY METHOD (cleaner syntax) ---
df.query('age > 28 and department == "Eng"')
df.query('age > 28 or department == "HR"')
# Why query()? More readable, no need for & | operators

# --- ISIN for multiple values ---
df[df['department'].isin(['Eng', 'Sales'])]
df[~df['department'].isin(['HR'])]  # NOT in list (~ negates)

# --- BETWEEN ---
df[df['age'].between(28, 35)]
# Equivalent to: (df['age'] >= 28) & (df['age'] <= 35)

# --- STR accessor for string filtering ---
df[df['name'].str.startswith('A')]
df[df['name'].str.contains('li')]
```

---

2. Sorting

```
# --- SORT BY SINGLE COLUMN ---
df.sort_values('salary')              # Ascending (default)
df.sort_values('salary', ascending=False)  # Descending

# --- SORT BY MULTIPLE COLUMNS ---
df.sort_values(['department', 'salary'], ascending=[True, False])
# Sort by department (A-Z), then salary (high-low) within each department

# --- SORT BY INDEX ---
df.sort_index()                    # Sort rows by index
df.sort_index(axis=1)                # Sort columns by name

# --- IN-PLACE SORTING ---
df.sort_values('salary', inplace=True)
# Why inplace? Saves memory (no copy), but modifies original
```

---

3. Column Operations

```
# --- ADD NEW COLUMN ---
df['bonus'] = df['salary'] * 0.10
df['age_group'] = pd.cut(df['age'], bins=[20, 30, 40, 50], labels=['20s', '30s', '40s'])
# pd.cut: discretize continuous data into bins
# Why cut? Create categories for analysis (age groups, salary bands)

df['salary_k'] = df['salary'] / 1000
df['is_senior'] = df['age'] > 30

# --- ASSIGN METHOD (chaining) ---
df = df.assign(
    bonus=lambda x: x['salary'] * 0.10,
```

```
    salary_k=lambda x: x['salary'] / 1000
)
# Why assign()? Enables method chaining, functional style

# --- DROP COLUMNS ---
df = df.drop(columns=['bonus'])
df.drop(columns=['bonus'], inplace=True)
df = df[['name', 'age', 'salary']]  # Select only specific columns

# --- RENAME COLUMNS ---
df = df.rename(columns={'name': 'employee_name', 'salary': 'annual_salary'})
# Why rename? Standardize column names, fix typos

# --- REORDER COLUMNS ---
df = df[['department', 'name', 'age', 'salary']]
# Why reorder? Logical grouping, presentation order

# --- INSERT COLUMN AT SPECIFIC POSITION ---
df.insert(1, 'employee_id', range(1, len(df)+1))
# insert(loc, column, value) -- inserts at position `loc`
```

---

4. Apply Functions

```
# --- apply: row/column-wise operations ---
df.apply(lambda x: x.mean())  # Mean of each numeric column
df.apply(lambda row: row['salary'] * 1.10, axis=1)  # Apply to each row
# Why axis=1? When you need values from multiple columns

# --- applymap: element-wise operations ---
df.applymap(lambda x: x.upper() if isinstance(x, str) else x)

# --- map: element-wise on Series ---
df['department'].map({'Eng': 'Engineering', 'Sales': 'Sales', 'HR': 'Human Resources'})
# Why map? Replace values using dict or Series (faster than apply for single column)

# --- WHEN TO USE WHAT? ---
# map(): Single column, value replacement (fastest)
# applymap(): All cells, same function (medium)
# apply(): Rows or columns, complex logic (slowest)
# Vectorized operations: Always prefer when possible (fastest)
```

Quick Reference

Task	**Code**
Label-based selection	`df.loc[row, col]`
Position-based selection	`df.iloc[row, col]`
Boolean filter	`df[df['col'] > 0]`
Query method	`df.query('col > 0')`
Multiple conditions	`df[(cond1) & (cond2)]`

Sort by column	`df.sort_values('col')`
Add column	`df['new'] = ...`
Drop column	`df.drop(columns=['col'])`
Rename column	`df.rename(columns={'old': 'new'})`
Apply function	`df.apply(func, axis=1)`

Next Steps

- **Module 04b:** GroupBy, merge, and concat
- **Module 05a:** Pivot tables and reshaping

MODULE-04b: DATA MANIPULATION -- GROUPBY, MERGE, CONCAT

---

5. GroupBy -- Split-Apply-Combine

The GroupBy pattern is the most powerful pandas operation.

The Three Steps

```
1. SPLIT: Divide data into groups based on some key
2. APPLY: Apply a function to each group independently
3. COMBINE: Combine results into a single DataFrame
```

Basic GroupBy

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank'],
    'age': [25, 30, 35, 28, 32, 40],
    'salary': [50000, 60000, 70000, 55000, 65000, 80000],
    'department': ['Eng', 'Sales', 'Eng', 'HR', 'Sales', 'Eng']
})

# --- AGGREGATION ---
df.groupby('department')['salary'].mean()
df.groupby('department')['salary'].agg(['mean', 'median', 'std', 'min', 'max', 'count'])

# Multiple aggregations per column
df.groupby('department').agg({
    'salary': ['mean', 'median', 'std', 'min', 'max'],
    'age': ['mean', 'min', 'max'],
    'name': 'count'
})

# --- WHY AGGREGATION? ---
# Summarize data by categories
# Compare groups
# Create summary reports
```

Transform and Filter

```
# --- TRANSFORM ---
# Apply function and return result with same index as input
df['dept_avg_salary'] = df.groupby('department')['salary'].transform('mean')
df['salary_diff_from_dept'] = df['salary'] - df['dept_avg_salary']
# Why transform? Keep original row structure while adding group-level info
```

```
# --- FILTER ---
# Keep only groups that meet a condition
df.groupby('department').filter(lambda x: len(x) >= 2)
# Why filter? Remove small groups, focus on significant segments

# --- APPLY (complex) ---
# Apply arbitrary function to each group
def normalize_salary(group):
    group['salary_normalized'] = (group['salary'] - group['salary'].mean()) / group['salary'].std()
    return group

df_normalized = df.groupby('department').apply(normalize_salary)
# Why apply? Complex operations that don't fit agg/transform
```

Advanced GroupBy Patterns

```
# --- MULTIPLE GROUPING COLUMNS ---
df.groupby(['department'])['salary'].mean()
# Creates MultiIndex (hierarchical index)

# --- NUNIQUE ---
df.groupby('department')['name'].nunique()
# Count unique values per group

# --- CUSTOM AGGREGATION ---
def range_func(x):
    return x.max() - x.min()

df.groupby('department')['salary'].agg(range_func)

# --- SIZE AND COUNT ---
df.groupby('department').size()     # Row count per group (includes NaN)
df.groupby('department').count()    # Non-null count per column
```

---

6. Merge and Join

Types of Joins

```
INNER JOIN: Only matching rows from both tables
LEFT JOIN:  All rows from left, matching from right (NULL if no match)
RIGHT JOIN: All rows from right, matching from left (NULL if no match)
OUTER JOIN: All rows from both tables (NULL where no match)
```

```
# --- SAMPLE DATA ---
employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4, 5],
    'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'dept_id': [101, 102, 101, 103, 102]
})

departments = pd.DataFrame({
    'dept_id': [101, 102, 103, 104],
    'dept_name': ['Engineering', 'Sales', 'HR', 'Marketing']
```

```
})

# --- INNER JOIN (default) ---
merged = pd.merge(employees, departments, on='dept_id')

# --- LEFT JOIN ---
merged = pd.merge(employees, departments, on='dept_id', how='left')

# --- RIGHT JOIN ---
merged = pd.merge(employees, departments, on='dept_id', how='right')

# --- OUTER JOIN ---
merged = pd.merge(employees, departments, on='dept_id', how='outer')

# --- JOIN ON DIFFERENT COLUMN NAMES ---
pd.merge(employees, departments, left_on='dept_id', right_on='dept_id')

# --- WHY MERGE vs JOIN? ---
# merge(): More flexible, explicit join type, works on any columns
# join(): Simpler syntax, joins on index by default
```

Merge Indicators

```
# Track which table each row came from
merged = pd.merge(employees, departments, on='dept_id', how='outer', indicator=True)
print(merged['_merge'])
# Values: 'left_only', 'right_only', 'both'
# Why indicator? Debug join issues, understand data overlap
```

---

7. Concat and Append

```
# --- VERTICAL CONCATENATION (stacking rows) ---
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})

result = pd.concat([df1, df2], ignore_index=True)
# ignore_index=True: Reset index in result
# Why concat? Combine datasets with same columns

# --- HORIZONTAL CONCATENATION (stacking columns) ---
df3 = pd.DataFrame({'C': [9, 10], 'D': [11, 12]})
result = pd.concat([df1, df3], axis=1)
# axis=1: Concatenate along columns
# Why horizontal concat? Add new columns from another source
```

Quick Reference

Task	Code

Group and aggregate	`df.groupby('col').agg({'val': 'mean'})`
Transform	`df.groupby('col')['val'].transform('mean')`
Filter groups	`df.groupby('col').filter(lambda x: len(x) > 2)`
Merge tables	`pd.merge(df1, df2, on='key')`
Left join	`pd.merge(df1, df2, on='key', how='left')`
Concatenate	`pd.concat([df1, df2])`

Next Steps

- **Module 05a:** Pivot tables and reshaping
- **Module 06a:** Basic visualization

MODULE-05a: DATA TRANSFORMATION -- PIVOT TABLES AND RESHAPING

---

1. Pivot Tables

Pivot tables summarize data by grouping and aggregating -- like Excel pivot tables.

```
import pandas as pd
import numpy as np

# Sample sales data
sales = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=100, freq='D'),
    'region': np.random.choice(['North', 'South', 'East', 'West'], 100),
    'product': np.random.choice(['Widget', 'Gadget', 'Tool'], 100),
    'sales': np.random.randint(100, 1000, 100),
    'quantity': np.random.randint(1, 50, 100),
    'discount': np.random.choice([0, 0.05, 0.10, 0.15, 0.20], 100)
})

# --- BASIC PIVOT TABLE ---
pivot = sales.pivot_table(
    values='sales',           # What to aggregate
    index='region',           # Row groups
    columns='product',        # Column groups
    aggfunc='mean',           # How to aggregate
    fill_value=0              # Fill missing combinations with 0
)
print(pivot)
# Why pivot_table? Create summary reports, compare categories

# --- MULTIPLE AGGREGATIONS ---
pivot = sales.pivot_table(
    values=['sales', 'quantity'],
    index='region',
    columns='product',
    aggfunc={'sales': ['mean', 'sum', 'count'], 'quantity': 'sum'},
    fill_value=0,
    margins=True,             # Add row/column totals (Grand Total)
    margins_name='Total'
)
# Why margins? See overall totals alongside breakdowns

# --- PIVOT vs GROUPBY ---
# GroupBy: Returns Series or DataFrame with index
# Pivot table: Returns DataFrame with both row and column indices
# Use pivot_table when you need a cross-tabular report format

# --- CROSSTAB ---
# Special case: frequency count of two categorical variables
ct = pd.crosstab(sales['region'], sales['product'])
print(ct)
# Why crosstab? Quick contingency table, chi-square analysis

# --- CROSSTAB WITH NORMALIZATION ---
ct_pct = pd.crosstab(sales['region'], sales['product'], normalize=True) * 100
```

```
print(ct_pct)
# Shows percentage of total
```

---

2. Reshaping: Wide ↔ Long

Wide to Long (Melting)

```
# Wide format: each month is a column
wide = pd.DataFrame({
    'product': ['A', 'B', 'C'],
    'Jan_2024': [100, 200, 150],
    'Feb_2024': [120, 180, 160],
    'Mar_2024': [110, 210, 140]
})

# --- MELT: Wide → Long ---
long = wide.melt(
    id_vars=['product'],          # Columns to keep as-is
    var_name='month',             # Name for the variable column
    value_name='sales'            # Name for the value column
)
print(long)
# Why melt? Most plotting libraries expect long format
# Long format is better for grouping and filtering

# --- STACK: MultiIndex columns → rows ---
df_multi = wide.set_index('product')
stacked = df_multi.stack()
# stack(): Move column level to row index
# unstack(): Move row level to column index
```

Long to Wide (Pivoting)

```
# --- PIVOT: Long → Wide ---
wide_back = long.pivot(index='product', columns='month', values='sales')
# Why pivot back? Create comparison tables, export to Excel

# --- PIVOT with aggregation (handles duplicates) ---
wide_agg = long.pivot_table(index='product', columns='month', values='sales', aggfunc='mean')
# Use pivot_table when there might be duplicate (index, column) combinations

# --- UNSTACK: GroupBy result → wide ---
grouped = sales.groupby(['region', 'product'])['sales'].mean()
wide = grouped.unstack()
# unstack(): Move innermost index level to columns
# Why unstack? Create readable reports from grouped data
```

Quick Reference

Task	Code
Pivot table	`df.pivot_table(values='v', index='i', columns='c', aggfunc='mea`
Crosstab	`pd.crosstab(df['a'], df['b'])`
Melt (wide→long)	`df.melt(id_vars=['a'], var_name='var', value_name='val')`
Pivot (long→wide)	`df.pivot(index='a', columns='b', values='c')`

Next Steps

- **Module 05b:** String operations, DateTime, time series
- **Module 06a:** Basic visualization

MODULE-05b: DATA TRANSFORMATION -- STRINGS, DATETIME, TIME SERIES

---

3. String Operations

The <code>.str</code> accessor provides vectorized string methods.

```
# Sample data with messy strings
df_str = pd.DataFrame({
    'name': ['  Alice Smith ', 'BOB JONES', 'charlie brown ', '  Diana Prince'],
    'email': ['ALICE@EXAMPLE.COM', 'bob@example.com', 'CHARLIE@Example.COM', 'diana@example.com'],
    'phone': ['(555) 123-4567', '555-987-6543', '(555) 456-7890', '555.321.6543']
})

# --- BASIC STRING METHODS ---
df_str['name_clean'] = df_str['name'].str.strip()          # Remove whitespace
df_str['name_lower'] = df_str['name'].str.lower()          # Lowercase
df_str['name_upper'] = df_str['name'].str.upper()          # Uppercase
df_str['name_title'] = df_str['name'].str.title()          # Title Case

# --- EXTRACTION ---
df_str['first_name'] = df_str['name_clean'].str.split().str[0]
df_str['last_name'] = df_str['name_clean'].str.split().str[-1]
df_str['email_domain'] = df_str['email'].str.extract(r'@(.+)$')
df_str['area_code'] = df_str['phone'].str.extract(r'[\(\-](\d{3})[\)\-\.]')

# --- REPLACEMENT ---
df_str['phone_clean'] = df_str['phone'].str.replace(r'[()\-\.\s]', '', regex=True)

# --- PATTERN MATCHING ---
print(df_str['email'].str.contains(r'@example\.com'))  # Contains pattern
print(df_str['email'].str.match(r'^[A-Z]'))            # Starts with uppercase

# --- SPLITTING ---
df_str[['first', 'last']] = df_str['name_clean'].str.split(' ', n=1, expand=True)
# n=1: Split only on first space
# expand=True: Return DataFrame instead of Series of lists

# --- LENGTH AND COUNT ---
df_str['name_length'] = df_str['name_clean'].str.len()
df_str['word_count'] = df_str['name_clean'].str.count(' ') + 1
```

---

4. DateTime Operations

```
# --- PARSING DATES ---
dates = pd.DataFrame({
    'date_str': ['2024-01-15', '01/20/2024', '2024-02-10', '15-Mar-2024'],
    'timestamp': ['2024-01-15 14:30:00', '2024-01-20 09:15:00', '2024-02-10 16:45:00', '2024-03-15 11:00:00']
})
```

```
dates['date'] = pd.to_datetime(dates['date_str'])
dates['timestamp'] = pd.to_datetime(dates['timestamp'])
# pd.to_datetime: Auto-detects most date formats
# format='%Y-%m-%d': Specify format for speed and accuracy

# --- EXTRACTING COMPONENTS ---
dates['year'] = dates['date'].dt.year
dates['month'] = dates['date'].dt.month
dates['day'] = dates['date'].dt.day
dates['day_of_week'] = dates['date'].dt.dayofweek      # 0=Monday, 6=Sunday
dates['day_name'] = dates['date'].dt.day_name()        # 'Monday', 'Tuesday', ...
dates['month_name'] = dates['date'].dt.month_name()    # 'January', 'February', ...
dates['quarter'] = dates['date'].dt.quarter            # 1, 2, 3, 4
dates['is_month_start'] = dates['date'].dt.is_month_start
dates['is_month_end'] = dates['date'].dt.is_month_end

# --- DATE ARITHMETIC ---
dates['days_since_epoch'] = (dates['date'] - pd.Timestamp('2024-01-01')).dt.days
dates['next_month'] = dates['date'] + pd.DateOffset(months=1)
dates['age'] = (pd.Timestamp.now() - dates['date']).dt.days / 365.25

# --- DATE RANGES ---
date_range = pd.date_range('2024-01-01', '2024-12-31', freq='D')
# freq='D': Daily, 'W': Weekly, 'M': Month-end, 'Q': Quarter-end, 'Y': Year-end
# freq='B': Business days, 'H': Hours, 'T' or 'min': Minutes

# --- BUSINESS DAYS ---
bday_range = pd.bdate_range('2024-01-01', periods=10)
# Business days only (excludes weekends)
```

---

5. Time Series Resampling

```
# --- CREATE TIME SERIES ---
np.random.seed(42)
ts = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=365, freq='D'),
    'sales': np.random.randint(100, 1000, 365).cumsum()
}).set_index('date')

# --- RESAMPLING ---
# Downsample: Daily → Monthly
monthly = ts['sales'].resample('ME').sum()
# 'ME': Month end (pandas 2.2+)
# aggfunc: sum(), mean(), first(), last(), ohlc()

# Downsample: Daily → Weekly
weekly = ts['sales'].resample('W').mean()

# Upsample: Monthly → Daily (with interpolation)
daily_from_monthly = monthly.resample('D').interpolate()
# Why interpolate? Fill gaps when upsampling

# --- ROLLING WINDOWS ---
# Moving average (smooth out noise)
ts['sales_7d_avg'] = ts['sales'].rolling(window=7).mean()
ts['sales_30d_avg'] = ts['sales'].rolling(window=30).mean()

# Rolling statistics
ts['sales_7d_std'] = ts['sales'].rolling(window=7).std()
ts['sales_7d_min'] = ts['sales'].rolling(window=7).min()
```

```
ts['sales_7d_max'] = ts['sales'].rolling(window=7).max()

# --- EXPANDING WINDOWS ---
# Cumulative statistics
ts['cumsum'] = ts['sales'].expanding().sum()
ts['cummean'] = ts['sales'].expanding().mean()
ts['cummax'] = ts['sales'].expanding().max()

# --- EXPONENTIAL WEIGHTED ---
# More weight to recent observations
ts['ewm_30'] = ts['sales'].ewm(span=30).mean()
# Why EWM? React faster to recent changes than simple moving average
```

---

6. Categorical Data

```
# --- CREATE CATEGORIES ---
df_cat = pd.DataFrame({
    'size': ['S', 'M', 'L', 'XL', 'S', 'M', 'L', 'XL'],
    'priority': ['High', 'Low', 'Medium', 'High', 'Low', 'Medium', 'High', 'Low']
})

# Convert to categorical
df_cat['size'] = df_cat['size'].astype('category')

# --- ORDERED CATEGORIES ---
size_order = pd.CategoricalDtype(categories=['S', 'M', 'L', 'XL'], ordered=True)
df_cat['size'] = df_cat['size'].astype(size_order)

priority_order = pd.CategoricalDtype(categories=['Low', 'Medium', 'High'], ordered=True)
df_cat['priority'] = df_cat['priority'].astype(priority_order)

# Why ordered? Enables meaningful sorting and comparison
print(df_cat.sort_values('size'))  # Sorts by category order, not alphabetically

# --- CATEGORY OPERATIONS ---
print(df_cat['size'].cat.categories)    # List categories
print(df_cat['size'].cat.codes)         # Numeric codes (0, 1, 2, ...)

# --- MEMORY SAVINGS ---
df_large = pd.DataFrame({'col': np.random.choice(['A', 'B', 'C', 'D', 'E'], 1_000_000)})
print(f"Object: {df_large['col'].memory_usage(deep=True)} bytes")
df_large['col_cat'] = df_large['col'].astype('category')
print(f"Category: {df_large['col_cat'].memory_usage(deep=True)} bytes")
# Category typically uses ~10-20% of object memory
```

Quick Reference

Task	Code
String strip	`df['col'].str.strip()`
String extract	`df['col'].str.extract(r'pattern')`

Parse dates	`pd.to_datetime(df['col'])`
Date components	`df['col'].dt.year`, `.dt.month`, `.dt.day`
Resample	`df.resample('M').sum()`
Rolling mean	`df['col'].rolling(7).mean()`
Categorical	`df['col'].astype('category')`

Next Steps

- **Module 06a:** Basic visualization
- **Module 07a:** Export to CSV and Excel

MODULE-06a: VISUALIZATION -- BASIC CHARTS

Why Visualization Matters

> *"The single most important thing in data analysis is to **look at your data**." -- John Tukey*

Visualization helps you:

- Spot patterns, trends, and outliers instantly
- Communicate findings to stakeholders
- Validate assumptions before modeling
- Create production-grade reports

Chart Selection Guide

Goal	Chart Type	When to Use
Show trend over time	Line chart	Continuous time series
Compare categories	Bar chart	Few categories (<10)
Show distribution	Histogram/KDE	Understanding spread
Show relationship	Scatter plot	Two numerical variables
Show composition	Stacked bar/Pie	Parts of a whole
Show comparison	Box plot	Distributions across groups
Show correlation	Heatmap	Many variable relationships
Show multiple metrics	Dashboard	Executive summary

---

1. Setup and Styling

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set professional style
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("deep")

# Global styling parameters
plt.rcParams.update({
    'font.size': 11,           # Base font size
    'axes.titlesize': 14,      # Title font size
    'axes.labelsize': 12,      # Axis label font size
    'figure.dpi': 150,         # Screen resolution
    'axes.spines.top': False,  # Remove top border
```

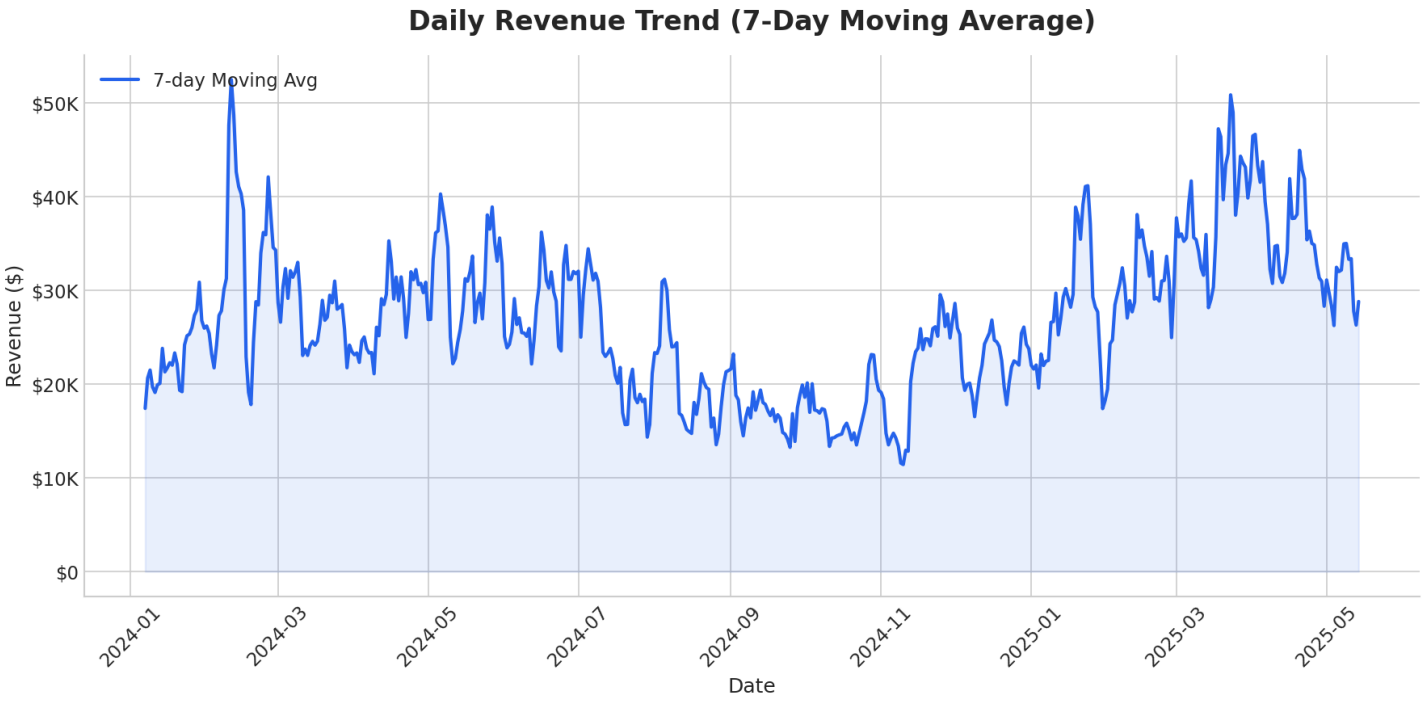
```
    'axes.spines.right': False,# Remove right border
})

# Why these settings?
# - seaborn style: Clean, professional look
# - No top/right spines: Reduces chart junk (Tufte's principle)
# - DPI 150: Sharp on screens and print
```

---

2. Line Charts -- Trends Over Time

```
# Sample time series
np.random.seed(42)
sales = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=365, freq='D'),
    'revenue': np.random.lognormal(mean=10, sigma=0.5, size=365)
})
sales['revenue'] *= 1 + 0.3 * np.sin(2 * np.pi * sales['date'].dt.dayofyear / 365)

# --- BASIC LINE CHART ---
fig, ax = plt.subplots(figsize=(12, 6))
daily = sales.groupby('date')['revenue'].sum()
ax.plot(daily.index, daily.values, color='#2563eb', linewidth=1.5)
ax.set_title('Daily Revenue', fontsize=16, fontweight='bold', pad=15)
ax.set_xlabel('Date')
ax.set_ylabel('Revenue ($)')
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('charts/line_basic.png', bbox_inches='tight')
plt.close()

# --- LINE WITH MOVING AVERAGE ---
fig, ax = plt.subplots(figsize=(12, 6))
# Raw daily data (faint)
ax.plot(daily.index, daily.values, color='#94a3b8', linewidth=0.8, alpha=0.5, label='Daily')
# 7-day moving average (prominent)
ma7 = daily.rolling(7).mean()
ax.plot(ma7.index, ma7.values, color='#2563eb', linewidth=2.5, label='7-Day MA')
# Fill under curve
ax.fill_between(ma7.index, ma7.values, alpha=0.1, color='#2563eb')
ax.set_title('Daily Revenue Trend (7-Day Moving Average)', fontsize=16, fontweight='bold', pad=15)
ax.set_xlabel('Date')
ax.set_ylabel('Revenue ($)')
ax.legend(loc='upper left')
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('charts/line_with_ma.png', bbox_inches='tight')
plt.close()
```

See generated chart:

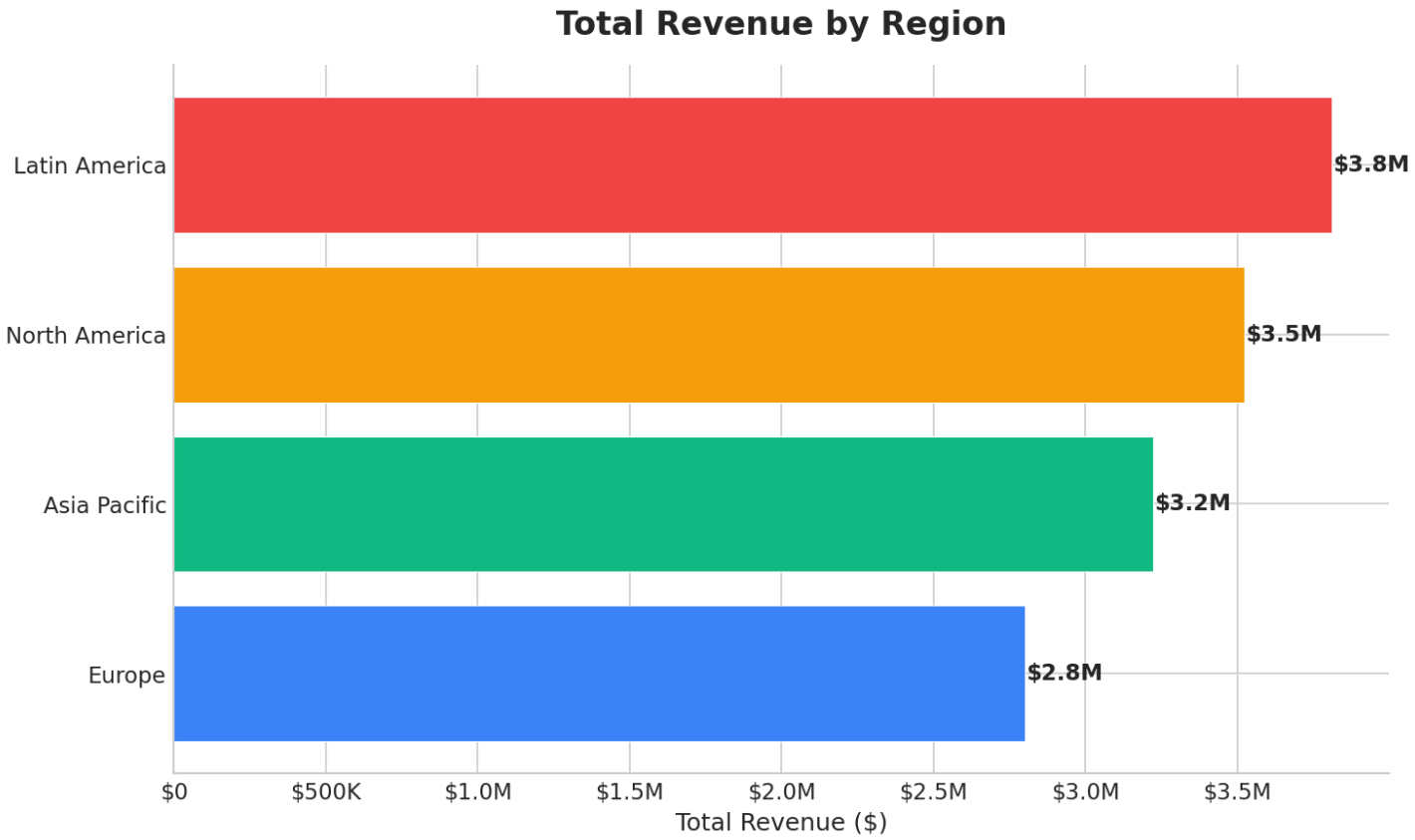


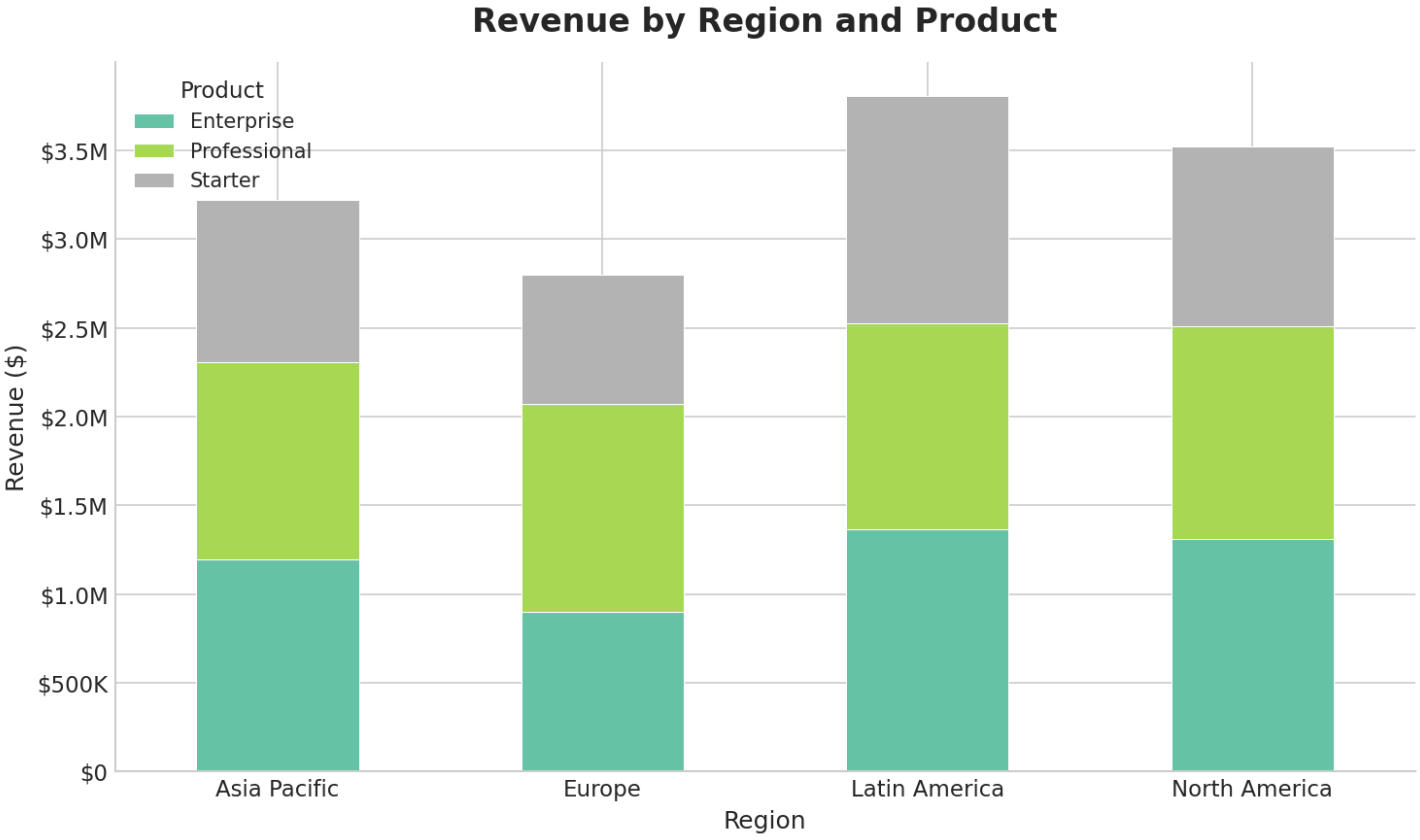
3. Bar Charts -- Comparing Categories

```
# --- SIMPLE BAR CHART ---
fig, ax = plt.subplots(figsize=(10, 6))
region_totals = sales.groupby('region')['revenue'].sum().sort_values(ascending=True)
colors = ['#3b82f6', '#10b981', '#f59e0b', '#ef4444']
bars = ax.barh(region_totals.index, region_totals.values, color=colors, edgecolor='white')
# Add value labels
for bar, val in zip(bars, region_totals.values):
    ax.text(bar.get_width() + 1000, bar.get_y() + bar.get_height()/2,
            f'${val/1e6:.1f}M', va='center', fontweight='bold')
ax.set_title('Total Revenue by Region', fontsize=16, fontweight='bold', pad=15)
ax.set_xlabel('Total Revenue ($)')
plt.tight_layout()
plt.savefig('charts/bar_horizontal.png', bbox_inches='tight')
plt.close()

# --- GROUPED BAR CHART ---
fig, ax = plt.subplots(figsize=(10, 6))
pivot = sales.groupby(['region', 'product'])['revenue'].sum().unstack()
pivot.plot(kind='bar', ax=ax, colormap='Set2', edgecolor='white', width=0.8)
ax.set_title('Revenue by Region and Product', fontsize=16, fontweight='bold', pad=15)
ax.set_xlabel('Region')
ax.set_ylabel('Revenue ($)')
ax.legend(title='Product')
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig('charts/bar_grouped.png', bbox_inches='tight')
plt.close()

# --- STACKED BAR CHART ---
fig, ax = plt.subplots(figsize=(10, 6))
pivot.plot(kind='bar', stacked=True, ax=ax, colormap='Set2', edgecolor='white')
ax.set_title('Revenue by Region and Product (Stacked)', fontsize=16, fontweight='bold', pad=15)
ax.set_xlabel('Region')
ax.set_ylabel('Revenue ($)')
```

```
ax.legend(title='Product')
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig('charts/bar_stacked.png', bbox_inches='tight')
plt.close()
```

See generated charts:

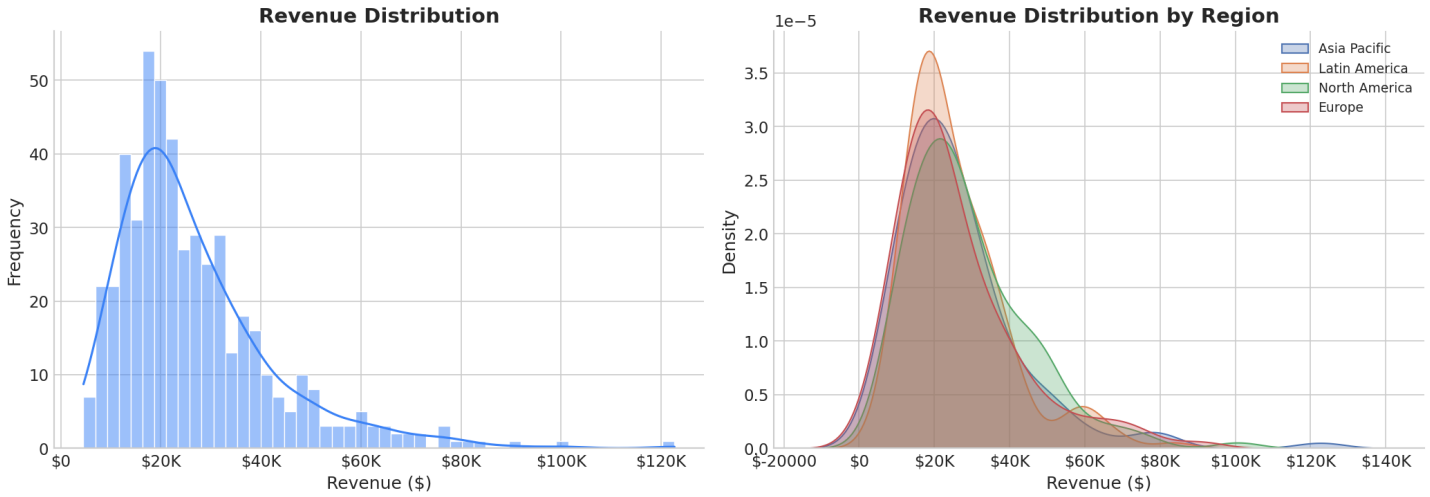




4. Histograms and Distribution Plots

```
# --- HISTOGRAM WITH KDE ---
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left: Histogram
sns.histplot(sales['revenue'], bins=50, kde=True, ax=axes[0], color='#3b82f6', edgecolor='white')
axes[0].set_title('Revenue Distribution', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Revenue ($)')
axes[0].set_ylabel('Frequency')

# Right: KDE by group
for region in sales['region'].unique():
    data = sales[sales['region'] == region]['revenue']
    sns.kdeplot(data, ax=axes[1], label=region, fill=True, alpha=0.3)
axes[1].set_title('Revenue Distribution by Region', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Revenue ($)')
axes[1].legend()

plt.tight_layout()
plt.savefig('charts/hist_kde.png', bbox_inches='tight')
plt.close()

# Why histogram vs KDE?
# Histogram: Shows actual count distribution (good for discrete bins)
# KDE: Smoothed density estimate (good for comparing distributions)
# Use both together for complete picture
```

See generated chart:



Quick Reference

Chart	Code	Best For
Line	`ax.plot(x, y)`	Trends over time
Bar	`ax.bar(x, y)` or `df.plot.bar()`	Category comparison

Histogram	`sns.histplot(data)`	Distribution
KDE	`sns.kdeplot(data)`	Smoothed distribution

Next Steps

- **Module 06b:** Scatter plots, box plots, heatmaps, dashboards
- **Module 07a:** Export to CSV and Excel

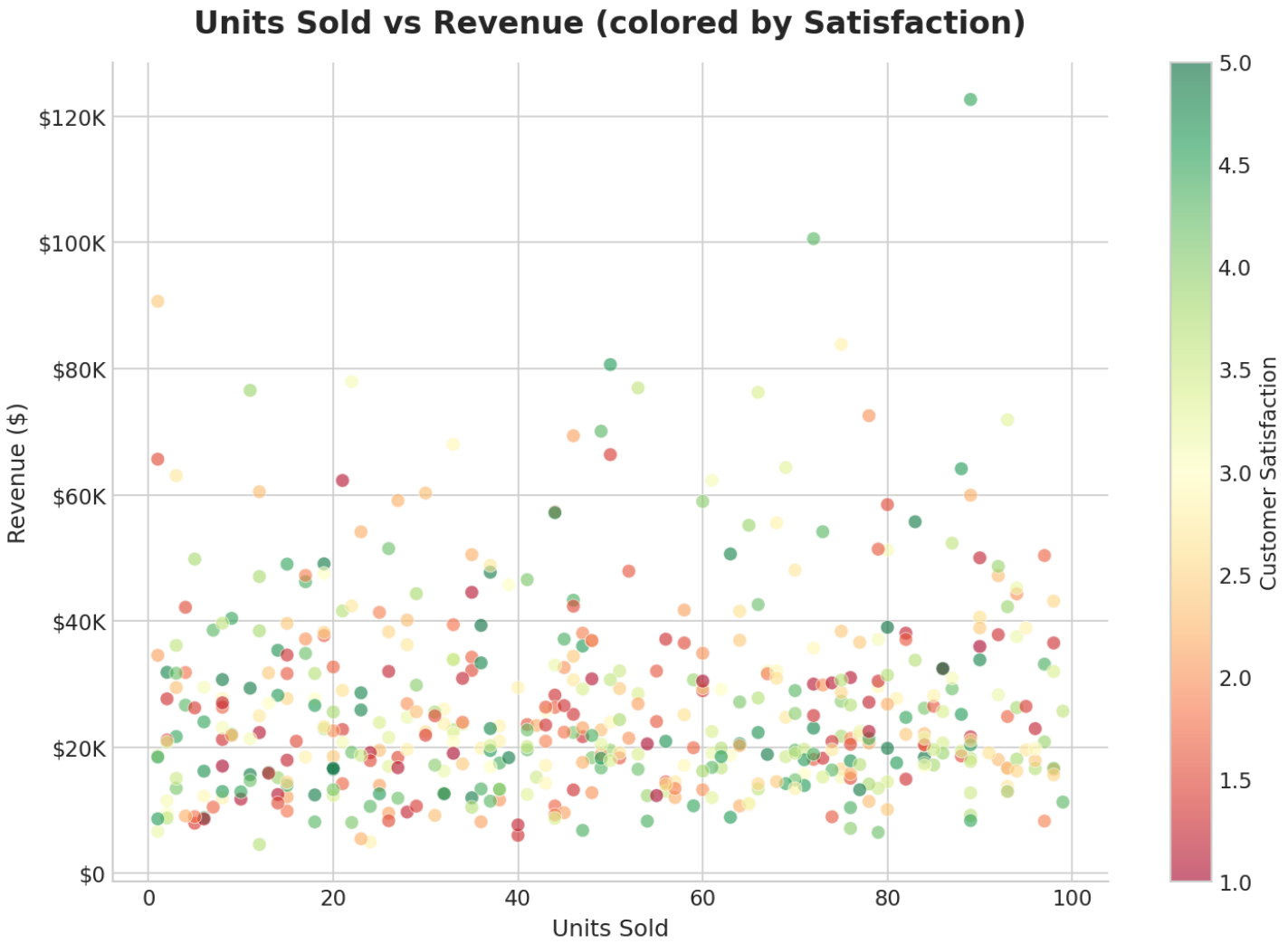
MODULE-06b: VISUALIZATION -- ADVANCED CHARTS AND DASHBOARDS

---

5. Scatter Plots -- Relationships

```
# --- BASIC SCATTER ---
fig, ax = plt.subplots(figsize=(10, 7))
ax.scatter(sales['units'], sales['revenue'], alpha=0.6, s=50, edgecolors='white', linewidth=0.5)
ax.set_title('Units vs Revenue', fontsize=16, fontweight='bold', pad=15)
ax.set_xlabel('Units Sold')
ax.set_ylabel('Revenue ($)')
plt.tight_layout()
plt.savefig('charts/scatter_basic.png', bbox_inches='tight')
plt.close()

# --- SCATTER WITH COLOR MAPPING ---
fig, ax = plt.subplots(figsize=(10, 7))
scatter = ax.scatter(
    sales['units'], sales['revenue'],
    c=sales['customer_satisfaction'],  # Color by satisfaction
    cmap='RdYlGn',                     # Red-Yellow-Green colormap
    alpha=0.6, s=50, edgecolors='white', linewidth=0.5
)
plt.colorbar(scatter, ax=ax, label='Customer Satisfaction')
ax.set_title('Units vs Revenue (colored by Satisfaction)', fontsize=16, fontweight='bold', pad=15)
ax.set_xlabel('Units Sold')
ax.set_ylabel('Revenue ($)')
plt.tight_layout()
plt.savefig('charts/scatter_colored.png', bbox_inches='tight')
plt.close()

# --- SCATTER WITH TREND LINE ---
fig, ax = plt.subplots(figsize=(10, 7))
sns.regplot(x='units', y='revenue', data=sales, ax=ax, scatter_kws={'alpha':0.5, 's':30},
            line_kws={'color': 'red', 'linewidth': 2})
ax.set_title('Units vs Revenue (with Regression Line)', fontsize=16, fontweight='bold', pad=15)
plt.tight_layout()
plt.savefig('charts/scatter_regression.png', bbox_inches='tight')
plt.close()
```

See generated chart:

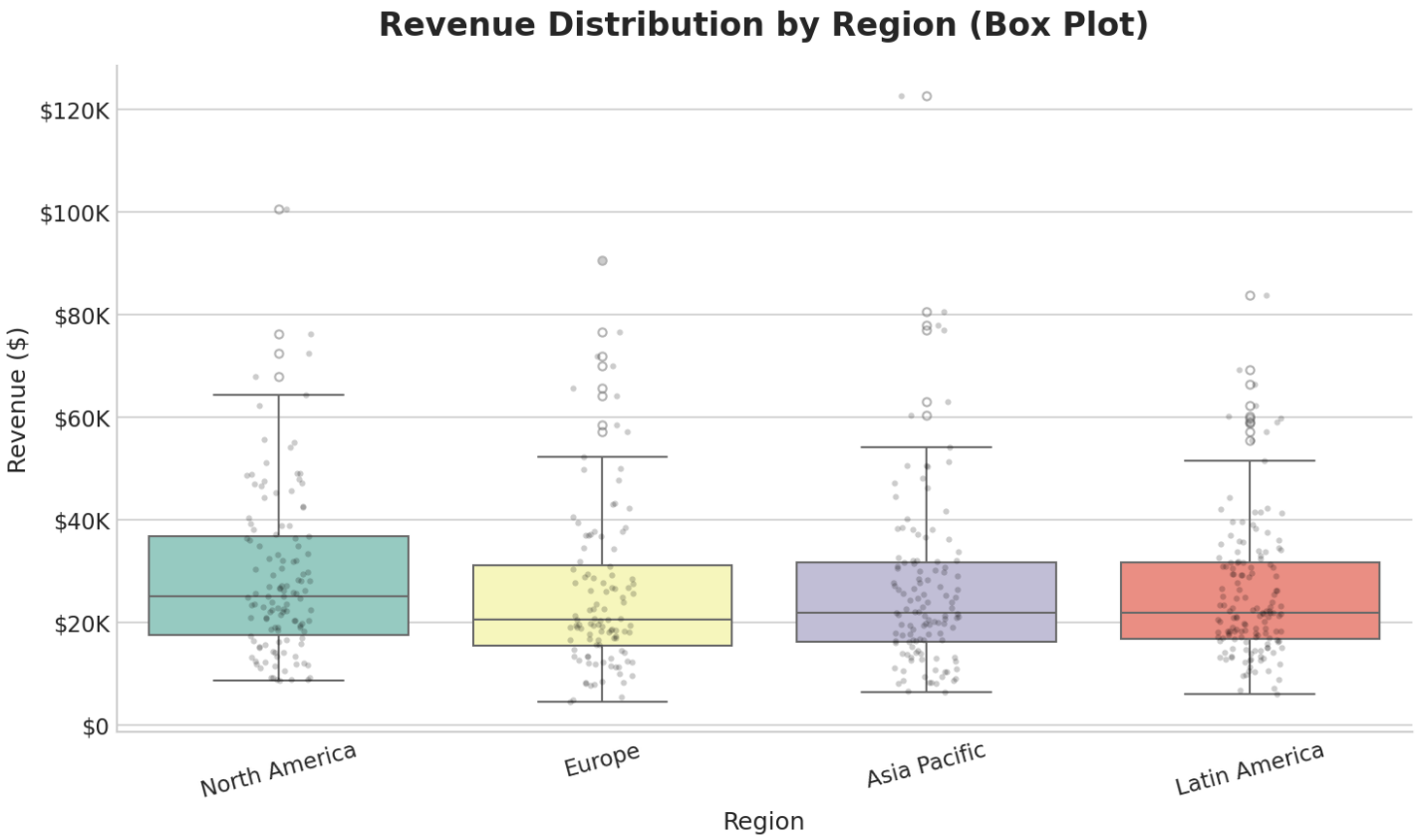


6. Box Plots -- Distribution Comparison

```
# --- BASIC BOX PLOT ---
fig, ax = plt.subplots(figsize=(10, 6))
sns.boxplot(data=sales, x='region', y='revenue', ax=ax, palette='Set3',
            flierprops=dict(marker='o', markersize=4, alpha=0.5))
# Add strip plot for individual points
sns.stripplot(data=sales, x='region', y='revenue', ax=ax, color='black', alpha=0.2, size=3)
ax.set_title('Revenue Distribution by Region', fontsize=16, fontweight='bold', pad=15)
ax.set_xlabel('Region')
ax.set_ylabel('Revenue ($)')
plt.xticks(rotation=15)
plt.tight_layout()
plt.savefig('charts/boxplot.png', bbox_inches='tight')
plt.close()

# --- VIOLIN PLOT (box + KDE) ---
fig, ax = plt.subplots(figsize=(10, 6))
sns.violinplot(data=sales, x='region', y='revenue', ax=ax, palette='pastel', inner='quartile', hue='region', legend=False)
ax.set_title('Revenue Distribution by Region (Violin Plot)', fontsize=16, fontweight='bold', pad=15)
plt.tight_layout()
plt.savefig('charts/violin.png', bbox_inches='tight')
plt.close()
```

```
# Why box plot vs violin?
# Box plot: Shows median, quartiles, outliers (compact)
# Violin plot: Shows full distribution shape (more informative)
# Use violin when distribution shape matters
```

See generated chart:



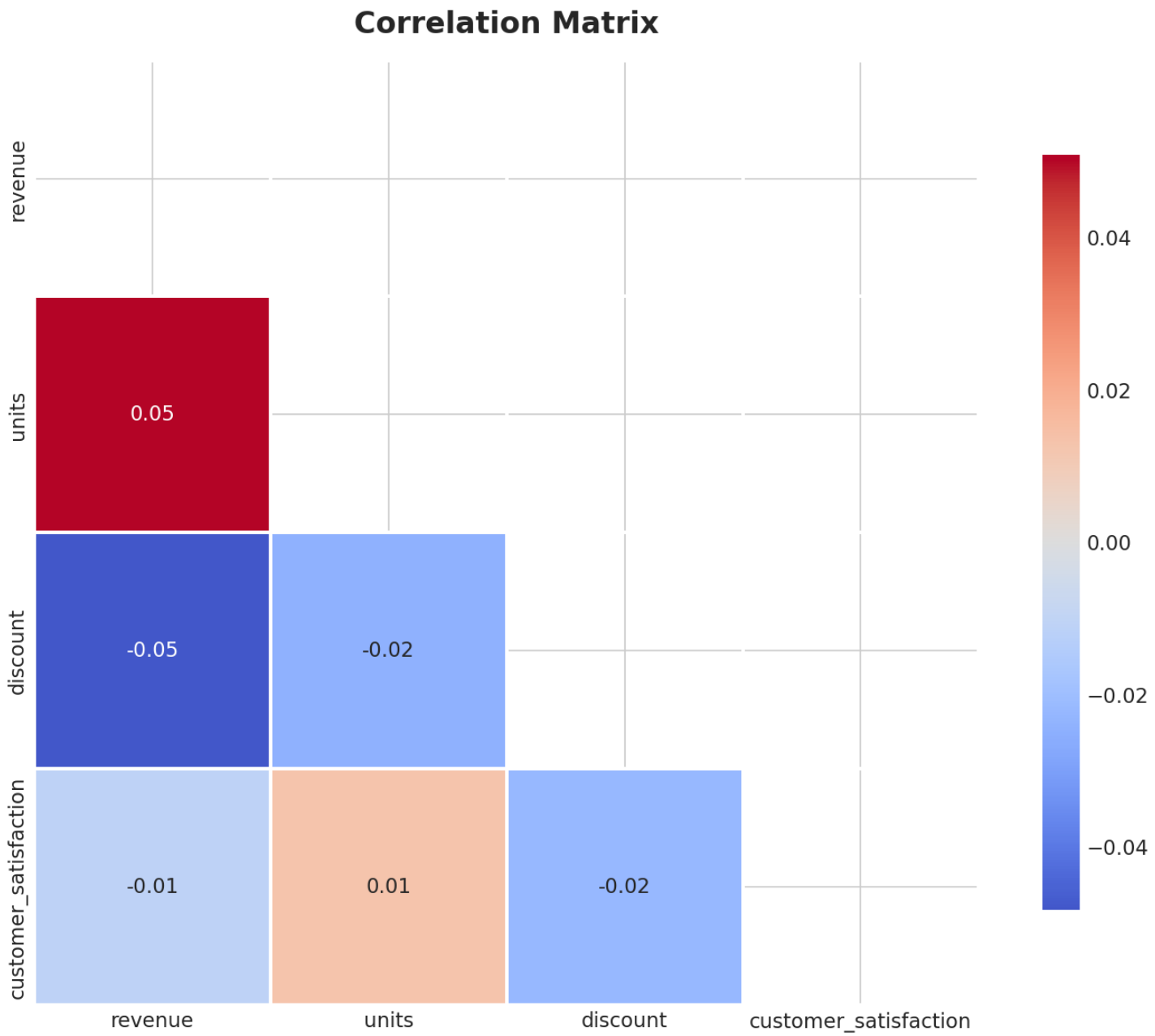
---

7. Heatmaps -- Correlation and Patterns

```
# --- CORRELATION HEATMAP ---
fig, ax = plt.subplots(figsize=(10, 8))
corr_cols = ['revenue', 'units', 'discount', 'customer_satisfaction']
corr_matrix = sales[corr_cols].corr()

# Mask upper triangle for cleaner look
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

sns.heatmap(
    corr_matrix,
    mask=mask,
    annot=True,          # Show correlation values
    fmt='.2f',           # Format to 2 decimal places
    cmap='coolwarm',     # Blue (negative) to Red (positive)
    center=0,            # Center colormap at 0
    square=True,         # Square cells
    linewidths=1,        # Cell borders
    cbar_kws={'shrink': 0.8},
    ax=ax,
    annot_kws={'fontsize': 11}
)
```

```
ax.set_title('Correlation Matrix', fontsize=16, fontweight='bold', pad=15)
plt.tight_layout()
plt.savefig('charts/heatmap.png', bbox_inches='tight')
plt.close()
```

See generated chart:

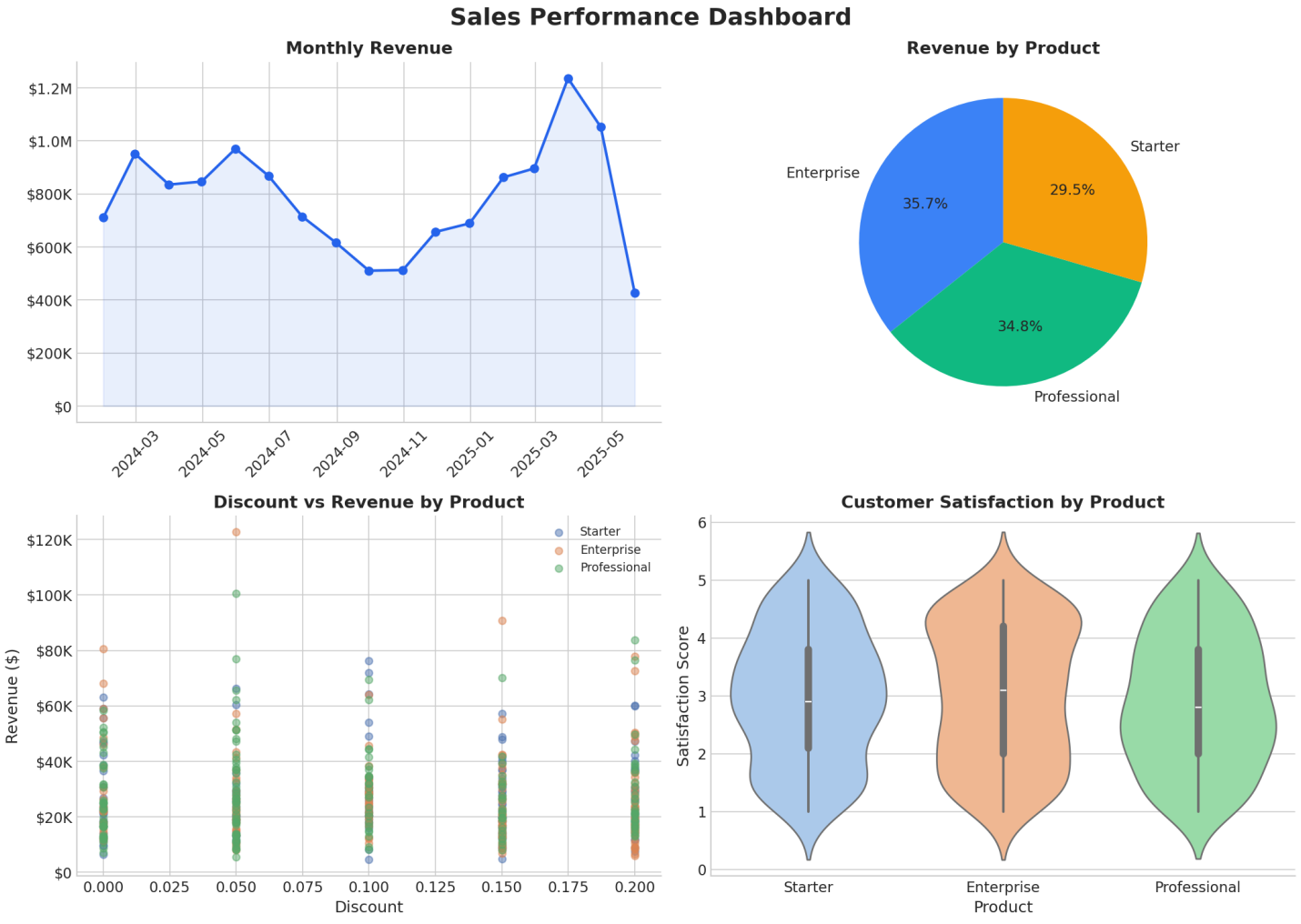


8. Multi-Panel Dashboards

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Sales Performance Dashboard', fontsize=18, fontweight='bold', y=0.98)

# Panel 1: Monthly revenue trend
monthly = sales.set_index('date')['revenue'].resample('ME').sum()
axes[0, 0].plot(monthly.index, monthly.values, color='#2563eb', linewidth=2, marker='o')
axes[0, 0].fill_between(monthly.index, monthly.values, alpha=0.1, color='#2563eb')
```

```
axes[0, 0].set_title('Monthly Revenue', fontsize=13, fontweight='bold')
axes[0, 0].tick_params(axis='x', rotation=45)

# Panel 2: Product mix pie chart
product_mix = sales.groupby('product')['revenue'].sum()
axes[0, 1].pie(product_mix.values, labels=product_mix.index, autopct='%1.1f%%',
               colors=['#3b82f6', '#10b981', '#f59e0b'], startangle=90, textprops={'fontsize': 11})
axes[0, 1].set_title('Revenue by Product', fontsize=13, fontweight='bold')

# Panel 3: Discount vs Revenue by product
for prod in sales['product'].unique():
    data = sales[sales['product'] == prod]
    axes[1, 0].scatter(data['discount'], data['revenue'], alpha=0.5, label=prod, s=30)
axes[1, 0].set_title('Discount vs Revenue by Product', fontsize=13, fontweight='bold')
axes[1, 0].set_xlabel('Discount')
axes[1, 0].set_ylabel('Revenue ($)')
axes[1, 0].legend(fontsize=9)

# Panel 4: Satisfaction by product
sns.violinplot(data=sales, x='product', y='customer_satisfaction', ax=axes[1, 1], hue='product', palette='pastel', legend=False)
axes[1, 1].set_title('Customer Satisfaction by Product', fontsize=13, fontweight='bold')
axes[1, 1].set_xlabel('Product')
axes[1, 1].set_ylabel('Satisfaction Score')

plt.tight_layout()
plt.savefig('charts/dashboard.png', dpi=150, bbox_inches='tight')
plt.close()
```

See generated chart:



9. Chart Customization -- Production Grade

```
# --- CUSTOM FORMATTERS ---
from matplotlib.ticker import FuncFormatter

def currency_fmt(x, pos):
    """Format numbers as currency."""
    if x >= 1e6: return f'${x/1e6:.1f}M'
    elif x >= 1e3: return f'${x/1e3:.0f}K'
    return f'${x:.0f}'

def pct_fmt(x, pos):
    """Format numbers as percentage."""
    return f'{x*100:.0f}%'

# Apply formatter
ax.yaxis.set_major_formatter(FuncFormatter(currency_fmt))
ax.xaxis.set_major_formatter(FuncFormatter(pct_fmt))

# --- CUSTOM COLORS AND COLORMAPS ---
# Sequential (ordered data)
sns.color_palette("Blues", 5)
sns.color_palette("YlOrRd", 5)

# Diverging (positive/negative)
sns.color_palette("coolwarm", 5)
sns.color_palette("RdBu_r", 5)

# Categorical (unordered categories)
sns.color_palette("Set1", 5)
sns.color_palette("Set2", 5)
sns.color_palette("Set3", 5)

# --- ADDING ANNOTATIONS ---
ax.annotate('Peak', xy=(peak_date, peak_value),
            xytext=(peak_date + 30, peak_value + 10000),
            arrowprops=dict(arrowstyle='->', color='red'),
            fontsize=12, color='red')
```

---

10. Saving Charts for Production

```
# --- SAVE FOR WEB ---
plt.savefig('chart_web.png', dpi=150, bbox_inches='tight')
# dpi=150: Good for screens
# bbox_inches='tight': Remove extra whitespace

# --- SAVE FOR PRINT ---
plt.savefig('chart_print.pdf', dpi=300, bbox_inches='tight')
# dpi=300: Print quality
# .pdf: Vector format (scales without quality loss)

# --- SAVE FOR PRESENTATION ---
plt.savefig('chart_ppt.png', dpi=200, bbox_inches='tight', transparent=False)
# Higher DPI for projection

# --- SAVE MULTIPLE FORMATS ---
for fmt in ['png', 'pdf', 'svg']:
    plt.savefig(f'chart.{fmt}', dpi=300 if fmt == 'png' else None, bbox_inches='tight')
```

Quick Reference

Chart	Code	Best For
Scatter	`ax.scatter(x, y)`	Relationship between 2 vars
Box plot	`sns.boxplot(x, y, data)`	Distribution by group
Heatmap	`sns.heatmap(data)`	Correlation/pattern matrix
Pie	`ax.pie(sizes)`	Parts of a whole
Dashboard	`plt.subplots(2, 2)`	Multi-metric overview

Next Steps

- **Module 07a:** Export to CSV and Excel
- **Module 08a:** Performance optimization

MODULE-07a: DATA EXPORT -- CSV AND EXCEL

Export Overview

After cleaning and analyzing data, you need to output it. Pandas provides <code>to_*</code> methods for each format:

Function	Format	Best For
`to_csv()`	CSV	Data exchange, imports into other systems
`to_excel()`	Excel	Business reports, sharing with non-technical u
`to_json()`	JSON	APIs, web applications, NoSQL
`to_sql()`	SQL database	Database storage
`to_parquet()`	Parquet	Big data, analytics pipelines

---

1. CSV Export

```
import pandas as pd

df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'salary': [50000, 60000, 70000],
    'hire_date': pd.to_datetime(['2020-01-15', '2021-03-20', '2019-11-10'])
})

# --- BASIC EXPORT ---
df.to_csv('output.csv')
                    # Default: comma-separated, includes index and header

# --- CRITICAL PARAMETERS ---
df.to_csv('output.csv',
    index=False,           # Don't write row numbers (almost always False)
    header=True,           # Write column names (default True)
    sep=',',               # Field separator (',' for CSV, '\t' for TSV)
    encoding='utf-8',      # Character encoding
    na_rep='',             # Replace NaN with empty string
    columns=['name', 'age'],  # Export only specific columns
    date_format='%Y-%m-%d', # Format for datetime columns
    float_format='%.2f',    # Format for float columns
    quoting=1,             # 0=MINIMAL, 1=ALL, 2=NONNUMERIC, 3=NONE
)

# --- WHY INDEX=FALSE? ---
# Row numbers are meaningless in exported data
# They cause issues when re-importing
# Almost always set index=False for data exchange

# --- EXPORT TO TSV ---
df.to_csv('output.tsv', sep='\t', index=False)
```

```
# --- APPEND TO EXISTING FILE ---
df.to_csv('output.csv', mode='a', header=False, index=False)
# mode='a': Append mode (default is 'w' for write/overwrite)
# header=False: Don't write header again
```

---

2. Excel Export -- Basic

```
# --- BASIC EXPORT ---
df.to_excel('output.xlsx', index=False)
                    # Uses openpyxl engine by default

# --- WITH PARAMETERS ---
df.to_excel('output.xlsx',
    index=False,
    sheet_name='Employees',   # Sheet name (default 'Sheet1')
    startrow=0,               # Upper-left cell row (0-based)
    startcol=0,               # Upper-left cell column (0-based)
    columns=['name', 'age', 'salary'],  # Specific columns
    header=['Employee Name', 'Age', 'Annual Salary'],  # Custom header names
    float_format='%.2f',
    date_format='YYYY-MM-DD',
    engine='openpyxl',        # For .xlsx files
    engine='xlsxwriter',      # Alternative engine (more formatting options)
)
```

---

3. Excel Export -- Professional Reports

Multi-Sheet Reports

```
# Create multiple DataFrames
summary = df.groupby('department').agg({
    'salary': ['mean', 'median', 'count'],
    'age': 'mean'
})

details = df[['name', 'department', 'salary', 'age']]

trends = df.set_index('hire_date').resample('ME')['salary'].sum()

# Write to multiple sheets
with pd.ExcelWriter('report.xlsx', engine='openpyxl') as writer:
    summary.to_excel(writer, sheet_name='Summary')
    details.to_excel(writer, sheet_name='Details', index=False)
    trends.to_excel(writer, sheet_name='Monthly Trends')
# Why ExcelWriter? Single file with multiple sheets
# Context manager ensures proper file closure
```

Styled Excel Reports (openpyxl)

```
from openpyxl.styles import Font, PatternFill, Alignment
```

```
with pd.ExcelWriter('styled_report.xlsx', engine='openpyxl') as writer:
    df.to_excel(writer, sheet_name='Employees', index=False)

    # Get the workbook and worksheet
    workbook = writer.book
    worksheet = writer.sheets['Employees']

    # --- HEADER STYLING ---
    header_font = Font(bold=True, color='FFFFFF', size=12)
    header_fill = PatternFill(start_color='2563eb', end_color='2563eb', fill_type='solid')
    header_alignment = Alignment(horizontal='center', vertical='center')

    for cell in worksheet[1]:
        cell.font = header_font
        cell.fill = header_fill
        cell.alignment = header_alignment

    # --- DATA STYLING ---
    data_font = Font(size=11)
    for row in worksheet.iter_rows(min_row=2, max_row=worksheet.max_row,
                                   min_col=1, max_col=worksheet.max_column):
        for cell in row:
            cell.font = data_font
            cell.alignment = Alignment(vertical='center')

    # --- ALTERNATING ROW COLORS ---
    even_fill = PatternFill(start_color='F8FAFC', end_color='F8FAFC', fill_type='solid')
    for row_num in range(2, worksheet.max_row + 1):
        if row_num % 2 == 0:
            for cell in worksheet[row_num]:
                cell.fill = even_fill

    # --- AUTO-FIT COLUMN WIDTHS ---
    for col in worksheet.columns:
        max_length = 0
        column = col[0].column_letter
        for cell in col:
            try:
                max_length = max(max_length, len(str(cell.value)))
            except:
                pass
        worksheet.column_dimensions[column].width = max_length + 2

    # --- ADD FILTER ---
    worksheet.auto_filter.ref = worksheet.dimensions

    # --- FREEZE TOP ROW ---
    worksheet.freeze_panes = 'A2'

print("Styled report saved to styled_report.xlsx")
```

Styled Excel Reports (xlsxwriter)

```
# xlsxwriter provides more built-in formatting
with pd.ExcelWriter('styled_xlsxwriter.xlsx', engine='xlsxwriter') as writer:
    df.to_excel(writer, sheet_name='Employees', index=False)

    workbook = writer.book
    worksheet = writer.sheets['Employees']

    # Define formats
    header_fmt = workbook.add_format({
```

```
    'bold': True,
    'font_color': 'FFFFFF',
    'bg_color': '#2563eb',
    'border': 1,
    'align': 'center',
    'valign': 'vcenter'
})

data_fmt = workbook.add_format({
    'border': 1,
    'align': 'right',
    'num_format': '#,##0.00'
})

money_fmt = workbook.add_format({
    'border': 1,
    'num_format': '$#,##0'
})

# Apply header format
for col_num, value in enumerate(df.columns):
    worksheet.write(0, col_num, value, header_fmt)

# Apply data formats
for row_num, row_data in enumerate(df.values, start=1):
    for col_num, value in enumerate(row_data):
        if df.columns[col_num] == 'salary':
            worksheet.write(row_num, col_num, value, money_fmt)
        else:
            worksheet.write(row_num, col_num, value, data_fmt)

# Auto-fit columns
worksheet.autofit()
print("xlsxwriter report saved")
```

Quick Reference

Task	Code
Export CSV	`df.to_csv('file.csv', index=False)`
Export TSV	`df.to_csv('file.tsv', sep='\t', index=False)`
Export Excel	`df.to_excel('file.xlsx', index=False)`
Multi-sheet Excel	`pd.ExcelWriter('file.xlsx')` with context manager
Append CSV	`df.to_csv('file.csv', mode='a', header=False)`
Custom float format	`df.to_csv('file.csv', float_format='%.2f')`
Custom date format	`df.to_csv('file.csv', date_format='%Y-%m-%d')`

Next Steps

- **Module 07b:** Export to JSON, SQL, Parquet
- **Module 08a:** Performance optimization

MODULE-07b: DATA EXPORT -- JSON, SQL, PARQUET, AND MORE

---

4. JSON Export

```
import pandas as pd

df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'salary': [50000, 60000, 70000]
})

# --- BASIC EXPORT ---
df.to_json('output.json')
                    # Default: 'columns' orientation

# --- ORIENTATIONS ---
df.to_json('output.json', orient='records', indent=2)
# orient options:
# 'columns':   {column: {index: value}}  (default)
# 'records':   [{column: value}, ...]     (most common for APIs)
# 'index':     {index: {column: value}}
# 'split':     {index: [...], columns: [...], data: [[...]]}
# 'table':     {schema: {...}, data: [...]}  (Pandas schema)
# 'values':    [[value, value], ...]

# --- WITH PARAMETERS ---
df.to_json('output.json',
    orient='records',       # Array of objects
    indent=2,               # Pretty-print with 2-space indent
    date_format='iso',      # ISO 8601 date format
    date_unit='s',          # Date unit: 's', 'ms', 'us', 'ns'
    force_ascii=False,      # Allow non-ASCII characters
    double_precision=10,    # Float precision
)
```

---

5. SQL Export

```
from sqlalchemy import create_engine

# Create database connection
engine = create_engine('sqlite:///output.db')
engine = create_engine('postgresql://user:pass@host:5432/dbname')
engine = create_engine('mysql+pymysql://user:pass@host:3306/dbname')

# --- BASIC EXPORT ---
df.to_sql('employees', engine, index=False)
                    # Creates table and inserts data

# --- WITH PARAMETERS ---
```

```
df.to_sql('employees', engine,
    if_exists='replace',    # 'fail', 'replace', 'append'
    index=False,
    index_label='emp_id',   # Name for index column
    chunksize=1000,         # Insert in chunks (for large data)
    dtype={                 # Specify column types
        'name': 'VARCHAR(100)',
        'age': 'INTEGER',
        'salary': 'DECIMAL(10,2)',
        'hire_date': 'DATE'
    },
    method='multi',         # 'multi': multi-row INSERT (faster)
)

# --- WHY CHUNKSIZE? ---
# Large inserts can timeout or run out of memory
# chunksize=1000: Insert 1000 rows at a time
# Trade-off: smaller = safer, larger = faster
```

---

6. Other Export Formats

```
# --- PARQUET (columnar, compressed) ---
df.to_parquet('output.parquet', engine='pyarrow')
# Why parquet?
# - Columnar storage (fast for analytical queries)
# - Compression (smaller files)
# - Preserves data types
# - Industry standard for big data

# --- FEATHER (fast binary) ---
df.to_feather('output.feather')
# Why feather?
# - Very fast read/write
# - Preserves types
# - Language-agnostic (Python, R, Julia)

# --- PICKLE (Python-specific) ---
df.to_pickle('output.pkl')
# Why pickle?
# - Fastest for Python-only workflows
# - Preserves ALL pandas types
# - NOT portable to other languages
# - Security risk with untrusted data

# --- HTML ---
df.to_html('output.html', index=False, border=1)
# Why HTML?
# - Embed in web pages
# - Email reports
# - Documentation

# --- LATEX (for papers) ---
df.to_latex('output.tex', index=False)
# Why LaTeX?
# - Academic papers
# - Professional documents
```

---

7. Production Export Pattern

```
import logging
from pathlib import Path

def export_data(df, output_path: str, **kwargs):
    """
    Production-grade data exporter with validation and logging.

    Args:
        df: DataFrame to export
        output_path: Output file path
        **kwargs: Additional arguments for to_* functions
    """
    path = Path(output_path)

    # Validate DataFrame
    if df.empty:
        raise ValueError("Cannot export empty DataFrame")

    # Create output directory if needed
    path.parent.mkdir(parents=True, exist_ok=True)

    # Determine format from extension
    ext = path.suffix.lower()
    exporters = {
        '.csv': lambda: df.to_csv(path, index=False, **kwargs),
        '.tsv': lambda: df.to_csv(path, sep='\t', index=False, **kwargs),
        '.json': lambda: df.to_json(path, orient='records', indent=2, **kwargs),
        '.xlsx': lambda: df.to_excel(path, index=False, **kwargs),
        '.parquet': lambda: df.to_parquet(path, **kwargs),
        '.pkl': lambda: df.to_pickle(path, **kwargs),
    }

    if ext not in exporters:
        raise ValueError(f"Unsupported format: {ext}")

    try:
        exporters[ext]()
        file_size = path.stat().st_size
        logging.info(f"Exported {len(df)} rows to {output_path} ({file_size:,} bytes)")
    except Exception as e:
        logging.error(f"Failed to export to {output_path}: {e}")
        raise

# Usage
export_data(df, 'output/reports/employees.csv')
export_data(df, 'output/reports/employees.xlsx')
export_data(df, 'output/reports/employees.json')
```

Quick Reference

Task	Code
Export JSON	`df.to_json('file.json', orient='records')`
Export SQL	`df.to_sql('table', engine)`
Export Parquet	`df.to_parquet('file.parquet')`
Export Feather	`df.to_feather('file.feather')`

Export HTML	`df.to_html('file.html')`
Export Pickle	`df.to_pickle('file.pkl')`

Next Steps

- **Module 08a:** Performance optimization
- **Module 08b:** Production patterns and best practices

MODULE-08a: PRODUCTION-GRADE PANDAS -- PERFORMANCE

Production vs Notebook

Notebook code works for exploration. Production code must be:

- **Fast** -- handles large datasets efficiently
- **Robust** -- handles errors gracefully
- **Maintainable** -- readable, documented, testable
- **Reproducible** -- same input → same output

---

1. Performance Optimization

Vectorization: The #1 Rule

```
import pandas as pd
import numpy as np
import time

# Create large dataset
np.random.seed(42)
df = pd.DataFrame({
    'a': np.random.randn(1_000_000),
    'b': np.random.randn(1_000_000),
    'c': np.random.choice(['A', 'B', 'C'], 1_000_000)
})

# --- SLOW: Python loop ---
start = time.time()
result = []
for i in range(len(df)):
    result.append(df['a'].iloc[i] + df['b'].iloc[i])
df['sum_loop'] = result
print(f"Loop: {time.time() - start:.4f}s")

# --- SLOW: apply ---
start = time.time()
df['sum_apply'] = df.apply(lambda row: row['a'] + row['b'], axis=1)
print(f"apply: {time.time() - start:.4f}s")

# --- FAST: Vectorized ---
start = time.time()
df['sum_vec'] = df['a'] + df['b']
print(f"Vectorized: {time.time() - start:.4f}s")

# Vectorized operations are 10-100x faster because:
# 1. No Python loop overhead
# 2. numpy operations execute in C
# 3. Memory access is sequential (cache-friendly)
```

When Vectorization Isn't Enough

```
# --- NUMBA FOR COMPLEX FUNCTIONS ---
from numba import jit
import numpy as np

@jit(nopython=True)
def complex_calc(a, b):
    """JIT-compiled function for complex math."""
    return np.sin(a) * np.cos(b) + np.exp(-a**2)

start = time.time()
df['numba_result'] = complex_calc(df['a'].values, df['b'].values)
print(f"Numba: {time.time() - start:.4f}s")
```

Chunking for Large Files

```
# --- PROCESS IN CHUNKS ---
def process_large_file(file_path, chunk_size=100_000):
    """Process file in chunks to manage memory."""
    results = []
    for chunk in pd.read_csv(file_path, chunksize=chunk_size):
        # Process each chunk
        filtered = chunk[chunk['status'] == 'active']
        aggregated = filtered.groupby('category').agg({
            'revenue': ['sum', 'mean', 'count']
        })
        results.append(aggregated)

    # Combine results
    final = pd.concat(results).groupby(level=0).agg({
        ('revenue', 'sum'): 'sum',
        ('revenue', 'mean'): 'mean',
        ('revenue', 'count'): 'sum'
    })
    return final

# Why chunking?
# - Files larger than RAM will crash
# - Process one chunk at a time
# - Aggregate incrementally
```

---

2. Memory Optimization

Check Current Memory Usage

```
# --- MEMORY PROFILE ---
print(df.memory_usage(deep=True))
# deep=True: Include actual object sizes (strings, etc.)

# --- TOTAL MEMORY ---
total_mb = df.memory_usage(deep=True).sum() / 1024**2
print(f"Total: {total_mb:.1f} MB")
```

Downcast Numerical Types

```
# --- DOWNCAST INTEGERS ---
# int64 → int8/int16/int32 (if values fit)
df['small_int'] = pd.to_numeric(df['small_int'], downcast='integer')
# int64 (8 bytes) → int8 (1 byte): 8x memory savings

# --- DOWNCAST FLOATS ---
# float64 → float32
df['float_col'] = pd.to_numeric(df['float_col'], downcast='float')
# float64 (8 bytes) → float32 (4 bytes): 2x memory savings

# --- SPECIFIC DOWNSCASTING ---
def downcast_dataframe(df):
    """Aggressively downcast all numerical columns."""
    for col in df.select_dtypes(include=['int64']).columns:
        df[col] = pd.to_numeric(df[col], downcast='integer')
    for col in df.select_dtypes(include=['float64']).columns:
        df[col] = pd.to_numeric(df[col], downcast='float')
    return df

df_optimized = downcast_dataframe(df)
```

Use Categorical for Low-Cardinality Strings

```
# --- BEFORE: object dtype ---
print(df['category_col'].dtype)  # object
print(df['category_col'].memory_usage(deep=True))  # ~large

# --- AFTER: category dtype ---
df['category_col'] = df['category_col'].astype('category')
print(df['category_col'].dtype)  # category
print(df['category_col'].memory_usage(deep=True))  # ~small

# Memory savings = (unique_values / total_values) * original_size
# For 1M rows with 5 unique values: ~200,000x savings
```

Optimize Data Loading

```
# --- LOAD ONLY WHAT YOU NEED ---
# Use columns
df = pd.read_csv('data.csv', usecols=['col1', 'col2', 'col3'])

# Use dtypes
df = pd.read_csv('data.csv', dtype={
    'id': 'int32',
    'category': 'category',
    'value': 'float32'
})

# Use parse_dates
df = pd.read_csv('data.csv', parse_dates=['date_col'])

# Why optimize at load time?
# - Less memory from the start
# - Faster loading (less data to process)
# - No need to convert later
```

Quick Reference

Optimization	Code	Impact
Vectorization	`df['a'] + df['b']`	10-100x faster
Downcast int	`pd.to_numeric(col, downcast='integer	2-8x less memory
Downcast float	`pd.to_numeric(col, downcast='float')`	2x less memory
Categorical	`col.astype('category')`	10-100x less memory
Chunking	`read_csv(chunksize=100000)`	Enables large file processing
usecols	`read_csv(usecols=['a','b'])`	Faster load, less memory

Next Steps

- **Module 08b:** Error handling, testing, best practices
- **Module 07a:** Export to CSV and Excel

MODULE-08b: PRODUCTION-GRADE PANDAS -- PATTERNS AND BEST PRACTICES

---

3. Error Handling Patterns

```
import logging
from pathlib import Path

def safe_load_and_process(file_path):
    """Production-grade data loading with error handling."""
    logging.basicConfig(level=logging.INFO)

    # 1. Validate file exists
    path = Path(file_path)
    if not path.exists():
        logging.error(f"File not found: {file_path}")
        return None

    # 2. Load with error handling
    try:
        df = pd.read_csv(file_path)
        logging.info(f"Loaded {len(df)} rows")
    except pd.errors.ParserError as e:
        logging.error(f"Parse error: {e}")
        return None
    except Exception as e:
        logging.error(f"Load error: {e}")
        return None

    # 3. Validate data
    if df.empty:
        logging.warning("Empty DataFrame loaded")
        return None

    required_cols = ['id', 'value']
    missing_cols = [c for c in required_cols if c not in df.columns]
    if missing_cols:
        logging.error(f"Missing columns: {missing_cols}")
        return None

    # 4. Process with validation
    try:
        df['value'] = pd.to_numeric(df['value'], errors='coerce')
        df = df.dropna(subset=['value'])
        logging.info(f"Processed {len(df)} valid rows")
    except Exception as e:
        logging.error(f"Processing error: {e}")
        return None

    return df
```

---

4. Testing Data Pipelines

```
# --- ASSERTIONS FOR DATA VALIDATION ---
def validate_dataframe(df):
    """Validate DataFrame meets expectations."""
    assert len(df) > 0, "DataFrame is empty"
    assert 'id' in df.columns, "Missing 'id' column"
    assert df['id'].is_unique, "Duplicate IDs found"
    assert df['value'].notna().all(), "Missing values in 'value'"
    assert (df['value'] >= 0).all(), "Negative values found"
    assert df['date'].is_monotonic_increasing, "Dates not sorted"
    print("✓ All validations passed")

# --- PROPERTY-BASED TESTING ---
def test_pipeline():
    """Test that pipeline preserves key properties."""
    # Load raw data
    raw = pd.read_csv('raw_data.csv')

    # Run pipeline
    cleaned = clean_data(raw)

    # Verify properties
    assert len(cleaned) <= len(raw), "Pipeline added rows"
    assert set(cleaned.columns) == set(raw.columns), "Columns changed"
    assert cleaned['id'].nunique() == len(cleaned), "Duplicates introduced"
    print("✓ Pipeline tests passed")
```

---

5. Reproducible Workflows

```
# --- SET RANDOM SEED ---
import numpy as np
np.random.seed(42)
pd.options.mode.copy_on_write = True  # pandas 2.0+

# --- VERSION PINNING ---
# requirements.txt:
# pandas==2.2.0
# numpy==1.26.0
# matplotlib==3.8.0
# seaborn==0.13.0

# --- REPRODUCIBLE DATE HANDLING ---
# Always specify timezone
df['date'] = pd.to_datetime(df['date'], utc=True)
# Always specify date format
df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d')

# --- LOGGING ---
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('pipeline.log'),
        logging.StreamHandler()
    ]
)

def run_pipeline():
    logging.info("Starting pipeline")
```

```
    df = load_data('input.csv')
    logging.info(f"Loaded {len(df)} rows")

    df = clean_data(df)
    logging.info(f"Cleaned: {len(df)} rows")

    df = transform_data(df)
    logging.info(f"Transformed: {len(df)} rows")

    export_data(df, 'output.csv')
    logging.info("Pipeline complete")
```

---

6. Best Practices Checklist

```
"""
PANDAS PRODUCTION CHECKLIST
============================

DATA LOADING:
☐ Use usecols to load only needed columns
☐ Specify dtypes to save memory
☐ Use parse_dates for date columns
☐ Handle encoding explicitly
☐ Use chunksize for large files

DATA CLEANING:
☐ Check for missing values before processing
☐ Validate data types after loading
☐ Use errors='coerce' for numeric conversion
☐ Handle duplicates explicitly
☐ Validate business rules (no negative ages, etc.)

DATA MANIPULATION:
☐ Prefer vectorized operations over apply
☐ Use .loc for label-based selection
☐ Use .iloc for position-based selection
☐ Chain operations when possible
☐ Use query() for complex filters

MEMORY MANAGEMENT:
☐ Downcast numerical types
☐ Use categorical for low-cardinality strings
☐ Drop unused columns early
☐ Use chunking for large datasets
☐ Monitor memory usage with memory_usage()

PERFORMANCE:
☐ Profile with %timeit or time
☐ Use vectorization first
☐ Use numba for complex functions
☐ Avoid iterrows() and itertuples()
☐ Use .values for numpy operations

VISUALIZATION:
☐ Set professional style (seaborn)
☐ Use appropriate chart types
☐ Add labels, titles, legends
☐ Format numbers (currency, percentages)
☐ Save in multiple formats (PNG, PDF)
```

```
EXPORT:
☐ Always use index=False for CSV/Excel
☐ Specify encoding (utf-8)
☐ Use appropriate date formats
☐ Validate output before sharing
☐ Log export details (rows, size)

GENERAL:
☐ Use copy() when modifying DataFrames
☐ Handle errors with try/except
☐ Log all operations
☐ Version pin dependencies
☐ Write tests for critical functions
"""
```

---

7. Common Pitfalls and Solutions

```
# --- PITFALL 1: SettingWithCopyWarning ---
# WRONG: Creates warning, may not modify original
df[df['age'] > 30]['salary'] = 100000

# RIGHT: Use .loc
df.loc[df['age'] > 30, 'salary'] = 100000

# --- PITFALL 2: Chained Indexing ---
# WRONG: Slow, may fail
df['A'][0] = 10

# RIGHT: Use .loc or .iloc
df.loc[0, 'A'] = 10
df.iloc[0, 0] = 10

# --- PITFALL 3: Comparing Floats ---
# WRONG: Floating point precision issues
df[df['value'] == 0.1]

# RIGHT: Use np.isclose
df[np.isclose(df['value'], 0.1)]

# --- PITFALL 4: Modifying While Iterating ---
# WRONG: Unpredictable behavior
for idx, row in df.iterrows():
    df.loc[idx, 'new_col'] = row['a'] + row['b']

# RIGHT: Vectorized
df['new_col'] = df['a'] + df['b']

# --- PITFALL 5: Large Memory Usage ---
# WRONG: Loading everything into memory
df = pd.read_csv('large_file.csv')  # May crash

# RIGHT: Chunk or use PyArrow
for chunk in pd.read_csv('large_file.csv', chunksize=100_000):
    process(chunk)
```

---

8. Copy-on-Write (pandas 2.0+)

```
# Enable copy-on-write (default in pandas 3.0+)
pd.options.mode.copy_on_write = True

# Why copy-on-write?
# - Prevents SettingWithCopyWarning
# - Makes DataFrame behavior more predictable
# - Slight performance cost but safer

# Before CoW (pandas < 2.0):
# df['new'] = value  # May modify original or copy

# After CoW (pandas >= 2.0 with CoW enabled):
# df['new'] = value  # Always creates new DataFrame
# Original is never modified unexpectedly
```

CONGRATULATIONS!

You've completed the comprehensive pandas guide. You now have knowledge from beginner to production-grade pandas usage.

What You've Learned

Module	Topics
00	Core objects (Series, DataFrame, Index)
01	Data ingestion (CSV, JSON, Excel, SQL, APIs)
02	Exploratory data analysis (EDA)
03	Data cleaning (missing values, duplicates, types)
04	Data manipulation (filtering, grouping, merging)
05	Data transformation (pivot, reshape, strings, time series)
06	Professional visualization (charts, dashboards)
07	Data export (CSV, Excel, JSON, SQL, Parquet)
08	Production patterns (performance, memory, best practices)

Resources

Resource	Link
Official Documentation	https://pandas.pydata.org/docs/
10 Minutes to Pandas	https://pandas.pydata.org/docs/user_guide/10min.html
Python for Data Analysis (Wes McKinney)	https://wesmckinney.com/book/
Python Data Science Handbook (Jake VanderPlas)	https://jakevdp.github.io/PythonDataScienceHandbook/
Pandas Cookbook	https://github.com/jvns/pandas-cookbook
Effective Pandas	https://github.com/mattharrison/effective_pandas
100 Pandas Puzzles	https://github.com/ajcr/100-pandas-puzzles